

Riemannian Manifold Hamiltonian Monte Carlo and Low-Rank Approximation for Gaussian Process Regression

Toavina Thierry Leong-Pock-Tsy (toavina@aims.ac.za)
African Institute for Mathematical Sciences (AIMS)

Supervised by: Prof. Ulrich Paquet
African Institute for Mathematical Sciences (AIMS), South Africa

11 June 2026

Submitted in partial fulfillment of a structured masters degree at AIMS South Africa



Abstract

Riemannian manifold Hamiltonian Monte Carlo (RMHMC) is a Markov chain Monte Carlo method that samples from a target distribution using local geometric information of that distribution. We argue that it can be used to more efficiently evaluate the expectation of any function under an intractable posterior compared to Hamiltonian Monte Carlo and Metropolis-Hastings. In this work, this posterior will be taken as the product of a Gaussian process prior and a Student's t-distributed set of observations.

Riemannian manifold Hamiltonian Monte Carlo has high computational complexity, scaling in the order of $O(N^3)$ where N is the dimension of the parameter space. In an attempt to reduce the cost of the algorithm, we also introduce a novel version, **low-rank Riemannian manifold Hamiltonian Monte Carlo**, which uses a low-rank approximation of RMHMC's metric tensor to make computations faster, reducing the complexity to $O(NM^2)$ with $M < N$ for rank- M approximation and $O(N)$ in the case of a rank-1 approximation. We will show that, although it is not fully developed yet, it can potentially be an improvement over Hamiltonian Monte Carlo while being more computationally efficient than Riemannian manifold Hamiltonian Monte Carlo.

The JAX implementations of all algorithms presented in this work are publicly available on GitHub: <https://github.com/Toavina00/mcmc>

Declaration

I, the undersigned, hereby declare that the work contained in this research project is my original work, and that any work done by others or by myself previously has been acknowledged and referenced accordingly.



Toavina Thierry Leong-Pock-Tsy, 11 June 2026

Contents

Abstract	i
1 Introduction	1
1.1 Related Work	1
1.2 Outline	2
2 Gaussian Process	3
2.1 Basic Formulation	3
2.2 GP Regression with Gaussian likelihood	4
2.3 GP Regression with non-Gaussian likelihood	6
3 Markov Chain Monte Carlo	9
3.1 Monte Carlo Methods	9
3.2 Metropolis-Hastings	11
3.3 Hamiltonian Monte Carlo	12
3.4 Riemannian manifold Hamiltonian Monte Carlo	17
3.5 Rank-1 RMHMC	22
3.6 MCMC diagnostic	29
4 Experiments	31
4.1 Preliminaries: Sampling from a 2D Gaussian	31
4.2 Experiment Setup	32
4.3 MH, HMC, RMHMC Results	33
4.4 Rank-1 RMHMC Results	38
4.5 Discussion	39
5 Conclusion	41
References	44

1. Introduction

Markov chain Monte Carlo (MCMC) methods are a class of algorithms designed to sample from probability distributions whose densities are known only up to a normalising constant, making them intractable. By generating samples from the target distribution, we can compute the expectation of functions under that distribution which cannot be done analytically. Among the most popular MCMC methods is the Metropolis-Hastings (MH) (Metropolis et al., 1953; Hastings, 1970) which samples from the distribution through random walks on the support of the distribution. Hamiltonian Monte Carlo (HMC) (Neal, 2012) improves upon MH by using Hamiltonian dynamics to explore the support of the distribution better, rather than the random walk of MH. Riemannian manifold Hamiltonian Monte Carlo (RMHMC) (Girolami and Calderhead, 2011) is an extension of HMC that uses local geometric information of the target distribution. This enables the sampler to adapt to the curvature of the posterior and explore the support of the distribution even more efficiently.

RMHMC is the most sophisticated of the three algorithms, but it is also considerably more expensive than the other two. As we will see in Chapter 3, it requires solving multiple linear systems and can require computing up to third-order derivatives. We will see that by using JAX (Bradbury et al., 2018) we can avoid most heavy computations through automatic differentiation, while its just-in-time compilation feature can greatly optimise the matrix operations it requires.

Although RMHMC is expensive to run, we claim that it can sample from a complex distribution with an intractable probability density function more efficiently than MH and HMC. We quantify such efficiency as the quality of the samples divided by the time needed to produce these samples. In our work, this distribution will be defined as a posterior distribution with a Gaussian process prior and a non-Gaussian likelihood.

Additionally, we also explored the possibility of optimising RMHMC through the concept of low-rank Riemannian manifold Hamiltonian Monte Carlo, which is a novel exploratory work that has the potential to reduce the computational cost of Riemannian manifold Hamiltonian Monte Carlo while still performing well. We will cover the motivation behind it while describing the technical details and how using JAX automatic differentiation allows us to develop such algorithms.

The contributions of this research will be the following:

- For the case study of posterior distributions that arise from Gaussian processes with heavy-tailed observations, we show that RMHMC is overall more performant than the other algorithms. However, HMC can be tuned in some cases to have performance that is very close to RMHMC while being more computationally efficient.
- To decrease RMHMC's computational complexity, we introduce a novel approach, **low-rank RMHMC**. Through initial experiments, we show that while is still not fully understood yet, it has some potential and might prove to be useful with enough development.

1.1 Related Work

Our work shares similarities with Paquet and Fraccaro (2018). This report presents a technical description of an efficient implementation of RMHMC for Gaussian processes. They motivate their work by claiming that RMHMC presents itself as an excellent candidate for sampling from a complex posterior with highly correlated variables. In particular, they provide a detailed implementation and pseudocode

for implementing RMHMC for Gaussian processes with a particular focus on GP classification, which is another example of a GP model with a non-Gaussian likelihood. Our implementation is more general as it can be more easily adapted for any posterior distribution. Furthermore, it uses more modern numerical tools, particularly automatic differentiation and just-in-time compilation via JAX, enabling its adaptability and optimisation for modern hardware.

Our implementation uses automatic differentiation to optimise RMHMC, which has also been investigated in [Hayakawa and Asai \(2025\)](#). In contrast to our work, their approaches to optimisation focus on determining how to best integrate automatic differentiation into the process, as opposed to our work, which investigates low-rank matrix operations. Their work proposes a RMHMC algorithm specifically optimised for hierarchical Gaussian process models by using automatic differentiation and optimising it according to the structure of the model.

When using HMC, we have to specify some hyperparameters, such as the step size or the number of the leapfrog steps. We will talk more about what these are in Chapter 3. The performance of HMC is very sensitive to those parameters, and in practice, instead of tuning them manually, we use the No-U-Turn Sampler (NUTS) ([Hoffman and Gelman, 2011](#)), which removes the need for tuning the number of leapfrog steps. In practice, we also have no need to implement this algorithm and instead use tools such as Stan ([Carpenter et al., 2017](#)) or NumPyro ([Phan et al., 2019](#)). Those tools are built and optimised for probabilistic modelling and inference with implementations of various algorithms and diagnostic utilities. NumPyro, like our implementations, uses JAX as its backend, which makes use of its functionalities like vectorisation or hardware acceleration to make its code run faster. If operational, our low-rank Riemannian manifold Hamiltonian Monte Carlo could be integrated into those libraries.

Low-rank approximation introduced in [Eckart and Young \(1936\)](#) has been widely used in many domains to reduce the computational complexity of a matrix operation. For Gaussian processes, it has been used in [Williams and Seeger \(2001\)](#) to decrease the cost of computations by approximating the covariance matrix using a low-rank matrix while reducing the errors resulting from using the approximation. We use the same techniques to accelerate RMHMC sampling by computing the low-rank approximation of the metric tensor and using the approximation in the operations. We will see more about this in Chapter 3.

As they lie at the core of this work, we will have one entire chapter (Chapter 2) allocated for reviewing Gaussian processes, covering Gaussian process regression as it is described in [Rasmussen and Williams \(2006\)](#) and another chapter (Chapter 3) reviewing Metropolis-Hastings ([Metropolis et al., 1953](#); [Hastings, 1970](#)), Hamiltonian Monte Carlo ([Neal, 2012](#)) and Riemannian manifold Hamiltonian Monte Carlo ([Girolami and Calderhead, 2011](#)).

1.2 Outline

In this work, we first take a look at Gaussian processes in Chapter 2. We will review the basic results that will motivate our interest in Gaussian process regression, leading to the definition of the posterior distribution that we will use for model comparison. In Chapter 3 we will briefly review Monte Carlo methods and then dive into the basics and technical details of our implementations of Metropolis-Hastings, Hamiltonian Monte Carlo, Riemannian manifold Hamiltonian Monte Carlo and Rank-1 Riemannian manifold Hamiltonian Monte Carlo. Finally, in Chapter 4 we will give the details of our experiments and also present and discuss the results.

2. Gaussian Process

To better understand the motivation for the experiments, we must first review some fundamentals of the Gaussian process as applied to regression problems. The content of this chapter is mainly derived from [Rasmussen and Williams \(2006\)](#).

We can formulate a supervised learning task as finding a mapping from a finite set of input-target pairs, which can be used to predict the target corresponding to a given input. This mapping is learnt such that each prediction is as close to the target as possible. This notion of closeness is generally defined through a loss function that differs depending on the task. For regression problems, a common choice would be to take the squared difference between the targets and the outputs of the map. Gaussian processes can be used as a way to find such mapping.

To introduce Gaussian processes, [Rasmussen and Williams](#) describe two ways of tackling a supervised learning problem. One approach is to first select a class of functions and then find the function within that class, the specific function that best fits the data. The authors illustrate this using the class of functions that are linear in the input. The second approach they describe is to first place a prior distribution over the space of possible functions, with higher probability given to the most likely functions. Additionally, through a Bayesian framework, we can update this prior using a likelihood derived from our observed data, producing a posterior distribution over functions. Gaussian processes provide a way to define such a prior by generalising the Gaussian probability distribution to the space of functions.

In the following sections, we will first give a formal definition of Gaussian processes and then talk about Gaussian regression in the case of both Gaussian and non-Gaussian likelihoods. We will see that with a Gaussian likelihood, the posterior will be Gaussian and the mean and covariance will admit a closed-form expression, whereas for non-Gaussian likelihoods, this is no longer guaranteed.

2.1 Basic Formulation

2.1.1 Definition (Gaussian process). A Gaussian process (GP) is defined as a collection of random variables such that any finite subset has a joint Gaussian distribution. It is completely specified by a mean function $m(\mathbf{x})$ and a covariance function $k(\mathbf{x}, \mathbf{x}')$ defined as:

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})], \quad k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[(f(\mathbf{x}) - m(\mathbf{x}))(f(\mathbf{x}') - m(\mathbf{x}'))], \quad (2.1.1)$$

and we write $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$ ([Rasmussen and Williams, 2006](#)).

From this definition, given any finite collection of input points $\{\mathbf{x}_n\}_{n \in [N]}$, the random vector

$$\mathbf{f} = (f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_N))^\top \quad (2.1.2)$$

is jointly Gaussian with mean components $m(\mathbf{X})_n = m(\mathbf{x}_n)$ and covariance matrix $k(\mathbf{X}, \mathbf{X})_{mn} = k(\mathbf{x}_m, \mathbf{x}_n)$ with $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)^\top$.

Both mean and covariance functions encode the assumption we make on the function we want to learn. The covariance function, in particular, allows us to incorporate the concept of similarity between two points. If we consider a time series as an example, which loosely speaking could be defined as a univariate function that takes a value over time, we could use the mean function to inform a seasonal pattern and the covariance function to specify how close its values are as a function of how close they are in time.

To be a valid covariance function, a function must be symmetric and positive semi-definite. This means that for every input $\mathbf{x}$ and $\mathbf{x}'$,

$$k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x}), \quad k(\mathbf{x}, \mathbf{x}') \geq 0. \quad (2.1.3)$$

This follows from the fact that a covariance function naturally satisfies both of these properties. Additionally, the covariance function is typically parameterised by a hyperparameter θ , which can be tuned in order to obtain a posterior distribution that provides a better fit to the observed data.

As we can see, we can think of Gaussian processes as defining a distribution of functions, meaning sampling from this distribution will give us a function. In the next section we will discuss how we can use GP for regression tasks through a Bayesian framework by using it to define a prior over the set of possible functions and then update this prior using likelihood defined from the data. If this likelihood is assumed to be Gaussian, we will see that we will get a Gaussian posterior. But in the case of non-Gaussian distributions, we usually get an intractable posterior.

2.2 GP Regression with Gaussian likelihood

We will now look at the standard Gaussian process regression where the likelihood is assumed to be Gaussian. Formally, we want to learn a predictor f from a dataset $\{(\mathbf{x}_n, y_n)\}_{n \in [N]}$ by assuming that each target can be written as

$$y_n = f(\mathbf{x}_n) + \epsilon_n \quad (2.2.1)$$

where each ϵ_i is an independent Gaussian noise with variance σ^2 . It follows that

$$\mathbf{y} \mid \mathbf{f}, \mathbf{X} \sim \mathcal{N}(\mathbf{f}, \sigma^2 \mathbf{I}) \quad (2.2.2)$$

where $\mathbf{f} = (f(\mathbf{x}_1), f(\mathbf{x}_2), \dots, f(\mathbf{x}_N))^\top$, $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)^\top$ and $\mathbf{y} = (y_1, y_2, \dots, y_N)^\top$.

For the prior distribution, we need to define the mean and covariance functions $m(\mathbf{x})$ and $k(\mathbf{x}, \mathbf{x}')$. We can then build the mean vector and covariance matrix of the random variable $\mathbf{f}$. It then follows that

$$\mathbf{f} \mid \mathbf{X} \sim \mathcal{N}(m(\mathbf{X}), k(\mathbf{X}, \mathbf{X})) \quad (2.2.3)$$

Combining the prior and likelihood will allow us to derive the posterior distribution of the function. The product of Gaussians being another Gaussian, the posterior distribution will be a Gaussian.

Having established the prior distribution and likelihood, we will now consider the task of making predictions. A reasonable choice to obtain a point estimate from a given input $\mathbf{x}^*$ is to take the mean of the posterior $f(\mathbf{x}^*) \mid \mathbf{y}, \mathbf{X}, \mathbf{x}^*$. Furthermore, we can estimate the uncertainty of the prediction by computing its variance. For Gaussian process regression with a Gaussian likelihood, we can derive the expression of the mean and covariance for an input.

Let us denote $\mathbf{X}^* = (\mathbf{x}_1^*, \mathbf{x}_2^*, \dots, \mathbf{x}_N^*)^\top$ the points we want to make predictions on using our function. We will denote the random variable obtained by using the function f on those points as $\mathbf{f}^* = (f(\mathbf{x}_1^*), f(\mathbf{x}_2^*), \dots, f(\mathbf{x}_N^*))^\top$. To derive the posterior distribution $\mathbf{f}^* \mid \mathbf{y}, \mathbf{X}, \mathbf{X}^*$, we first write the joint distribution $\mathbf{y}, \mathbf{f}^* \mid \mathbf{X}, \mathbf{X}^*$. We know that $\mathbf{f}^* \mid \mathbf{X}^*$ will be Gaussian, and $\mathbf{y} \mid \mathbf{X}$, which is obtained by marginalising the Gaussian likelihood with the Gaussian prior $\mathbf{f} \mid \mathbf{X}$, will also be Gaussian. The joint distribution will then be Gaussian, and we will have

$$\mathbf{y}, \mathbf{f}^* \mid \mathbf{X}, \mathbf{X}^* \sim \mathcal{N} \left(\begin{bmatrix} m(\mathbf{X}) \\ m(\mathbf{X}^*) \end{bmatrix}, \begin{bmatrix} k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I} & k(\mathbf{X}, \mathbf{X}^*) \\ k(\mathbf{X}^*, \mathbf{X}) & k(\mathbf{X}^*, \mathbf{X}^*) \end{bmatrix} \right) \quad (2.2.4)$$

We can then obtain the posterior by deriving the conditional distribution, which gives us the following expressions for the mean and covariance of the Gaussian posterior.

$$E[\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*] = m(\mathbf{X}^*) + k(\mathbf{X}^*, \mathbf{X}) \left(k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I} \right)^{-1} (\mathbf{y} - m(\mathbf{X})), \quad (2.2.5)$$

$$\text{Cov}(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*) = k(\mathbf{X}^*, \mathbf{X}^*) - k(\mathbf{X}^*, \mathbf{X}) \left(k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I} \right)^{-1} k(\mathbf{X}, \mathbf{X}^*) \quad (2.2.6)$$

As mentioned in the previous section, we often parameterise the covariance function with a hyperparameter θ that we can tune to get a better fit on our dataset. Hyperparameter tuning in Gaussian process regression is done by finding the best hyperparameter that maximises the marginal likelihood. As we also mentioned, for a Gaussian likelihood, the marginal distribution is Gaussian, and we have an expression for the function we want to maximise. In practice, we would want to maximize the log marginal likelihood

$$\begin{aligned} \log p(\mathbf{y} | \mathbf{X}, \theta) = & -\frac{1}{2} (\mathbf{y} - m(\mathbf{X}))^\top (k_\theta(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I})^{-1} (\mathbf{y} - m(\mathbf{X})) \\ & - \frac{1}{2} \log |k_\theta(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}| - \frac{N}{2} \log 2\pi \end{aligned} \quad (2.2.7)$$

In the above expression, we explicitly denote the covariance function as $k_\theta(\mathbf{x}, \mathbf{x}')$ to emphasise that it depends on the hyperparameter θ . This expression can then be evaluated for multiple values of θ to perform a hyperparameter search. Figure 2.2 illustrates the effect of hyperparameter tuning in Gaussian process regression, by showing the resulting predictive mean for multiple values of θ when the covariance function is set to be a Radial Basis Function kernel (see e.g., [Rasmussen and Williams, 2006](#))

$$k_\theta(x, x') = \exp(-\theta(x - x')^2) \quad (2.2.8)$$

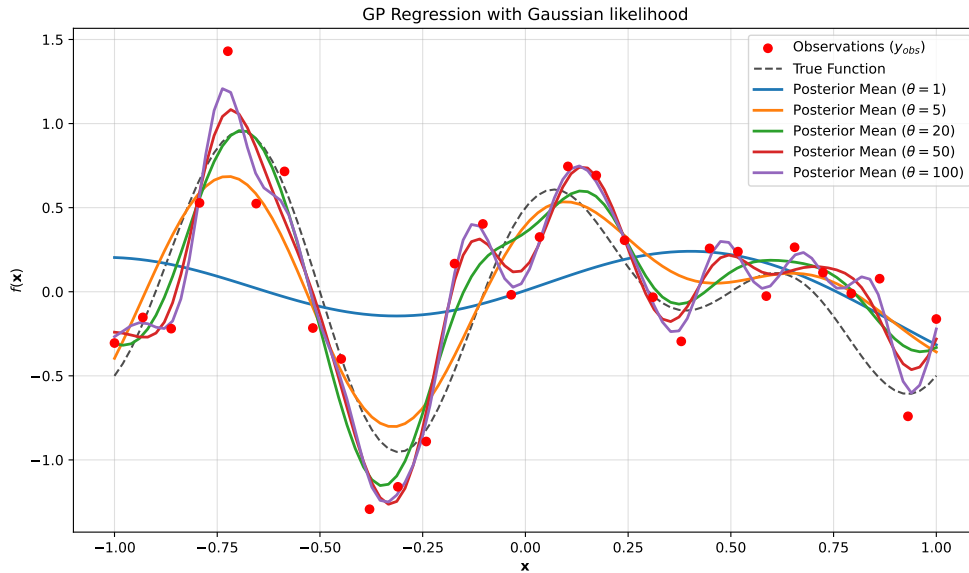


Figure 2.1: Gaussian process regression and the predictive means: the observations are generated using the true function by adding Gaussian noise to some of its values, and then multiple GP models with covariance defined in Equation (2.2.8) are fitted to the data with varying hyperparameters. We notice that the model is not able to fit the true function for small value of θ and for larger values it appears to overfit the data points.

We can see that when the likelihood is Gaussian, the posterior will also be Gaussian, and we have an expression for the mean and covariance. Furthermore, we can derive an expression for the marginal likelihood for hyperparameter tuning. This Gaussian assumption, however, greatly limits Gaussian process regression modelling. In real-world applications, data frequently contains rare extreme observations or outliers, and using the Gaussian assumption would make the model extremely sensitive to outliers. A more heavy-tailed noise distribution, such as Cauchy or Student's t-distribution, assigns a higher probability to values far from the mean than the Gaussian distribution, allowing it to capture outliers as low-probability events. In the next section, we will cover those Gaussian process regression models where the likelihood is non-Gaussian.

2.3 GP Regression with non-Gaussian likelihood

In the scope of this work, we will now consider extending Gaussian process regression beyond this Gaussian likelihood assumption. Formally, for a dataset $\{(\mathbf{x}_n, y_n)\}_{n \in [N]}$, we assume the following:

$$y_n = f(\mathbf{x}_n) + \eta_n \quad (2.3.1)$$

where each η_n is still an independent noise term but with a distribution that is not Gaussian. In the context of this work, we will assume that the noise has a Student's t-distribution.

2.3.1 Definition (Student's t-distribution). The probability density function of the Student's t-distribution is defined as:

$$p(\eta) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu s^2} \Gamma(\nu/2)} \left(1 + \frac{(\eta - \mu)^2}{\nu s^2} \right)^{-(\nu+1)/2}, \quad (2.3.2)$$

where Γ is the gamma function, μ is the centre of the distribution, s is a scale parameter and ν represents the degrees of freedom parameter. As $\nu \rightarrow \infty$, the Student's t-distribution approaches the Gaussian distribution with mean μ and standard deviation s . (MacKay, 2003)

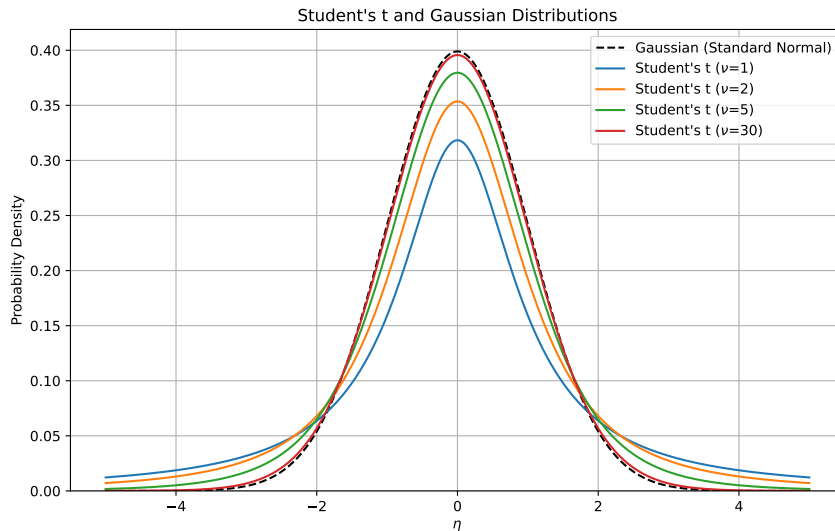


Figure 2.2: Student's t-distribution with $\mu = 0$, $s = 1$ at $\nu = 1, 2, 5, 30$ and $\nu \rightarrow \infty$ (Standard normal Gaussian)

Figure 2.2 shows that the Student's t-distribution has a heavier tail than a Gaussian, which increases as ν decreases. This justifies its application in robust regression. In our case, it follows that

$$y_n | f_n \sim \text{Student-t}(\mu = f_n, s, \nu) \quad (2.3.3)$$

where $f_n = f(\mathbf{x}_n)$.

Similarly to the case of Gaussian likelihood, in order to make predictions using the model, we would need to compute the mean and covariance of the resulting posterior given a set of inputs. However, there is no known closed-form expression for the mean and covariance of the posterior distribution resulting from the product of a Gaussian prior and Student's t likelihood for a finite degree of freedom. This lack of an analytic solution motivates the use of Monte Carlo methods, which will be introduced in Chapter 3. Those methods can be used to sample from a distribution and estimate expectations using those samples.

In order to use those methods, we would only need to be able to compute an unnormalised probability density function of the posterior in order to sample from it and hence estimate its mean or covariance. We can obtain such a function using Bayes' theorem and discard the normalisation term.

$$\begin{aligned} p(\mathbf{f} | \mathbf{y}, \mathbf{X}) &\propto \tilde{p}(\mathbf{f} | \mathbf{y}, \mathbf{X}) \\ &= p(\mathbf{y} | \mathbf{f}, \mathbf{X}) p(\mathbf{f} | \mathbf{X}) \\ &= \prod_{n=1}^N p(y_n | f_n) p(\mathbf{f} | \mathbf{X}) \end{aligned} \quad (2.3.4)$$

where $\tilde{p}$ corresponds to the unnormalised probability density function. This expression is valid for any choice of the noise distribution of η_n .

In order to make predictions from a set of inputs, $\mathbf{X}^*$ we would need to compute an expectation over the posterior $\mathbf{f} | \mathbf{y}, \mathbf{X}$.

$$\begin{aligned} p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*) &= \int p(\mathbf{f}^* | \mathbf{f}, \mathbf{y}, \mathbf{X}, \mathbf{X}^*) p(\mathbf{f} | \mathbf{y}, \mathbf{X}) d\mathbf{f} \\ &= \int p(\mathbf{f}^* | \mathbf{f}, \mathbf{X}, \mathbf{X}^*) p(\mathbf{f} | \mathbf{y}, \mathbf{X}) d\mathbf{f} \\ &= E_{\mathbf{f} | \mathbf{y}, \mathbf{X}} [p(\mathbf{f}^* | \mathbf{f}, \mathbf{X}, \mathbf{X}^*)] \end{aligned} \quad (2.3.5)$$

where $E_{\mathbf{f} | \mathbf{y}, \mathbf{X}}$ denotes the expectation over the posterior distribution $\mathbf{f} | \mathbf{y}, \mathbf{X}$. The above expression relies on the fact that each y_n is conditionally independent with every variable $f(\mathbf{x}_m)$ with $m \neq n$ when conditioned on $f(\mathbf{x}_n)$ (Rasmussen and Williams, 2006). The conditional distribution $\mathbf{f}^* | \mathbf{f}, \mathbf{X}, \mathbf{X}^*$ is Gaussian, as it is the obtained by conditioning a Gaussian with another Gaussian. We have the following expression for its mean and covariance:

$$E[\mathbf{f}^* | \mathbf{f}, \mathbf{X}, \mathbf{X}^*] = m(\mathbf{X}^*) + k(\mathbf{X}^*, \mathbf{X})k(\mathbf{X}, \mathbf{X})^{-1}(\mathbf{f} - m(\mathbf{X})) \quad (2.3.6)$$

$$\text{Cov}(\mathbf{f}^* | \mathbf{f}, \mathbf{X}, \mathbf{X}^*) = k(\mathbf{X}^*, \mathbf{X}^*) - k(\mathbf{X}^*, \mathbf{X})k(\mathbf{X}, \mathbf{X})^{-1}k(\mathbf{X}, \mathbf{X}^*) \quad (2.3.7)$$

We would also need a way to tune the hyperparameters θ of the covariance. This can still be done by maximising the marginal likelihood, although, in our case, we would not have a closed-form expression of the objective function anymore. However, we could still evaluate this objective function given a value

of the hyperparameter using Monte Carlo methods. This is due to the fact that we can rewrite the expression of the marginal likelihood as an expectation over the prior distribution.

$$p(\mathbf{y} | \mathbf{X}, \theta) = \int p(\mathbf{y} | \mathbf{f}, \mathbf{X}, \theta) p(\mathbf{f} | \mathbf{X}, \theta) d\mathbf{f} = E_{\mathbf{f} | \mathbf{X}, \theta} [p(\mathbf{y} | \mathbf{f}, \mathbf{X}, \theta)] \quad (2.3.8)$$

where $E_{\mathbf{f} | \mathbf{X}, \theta}$ denote the expectation over the prior distribution $\mathbf{f} | \mathbf{X}, \theta$. Furthermore, as we will see in Chapter 3, we can sample from a Gaussian efficiently, and this will allow us to estimate the objective function using M samples $\{\mathbf{f}^{(m)}\}_{m \in [M]}$ from the prior distribution as follows

$$p(\mathbf{y} | \mathbf{X}, \theta) \approx \frac{1}{M} \sum_{m=1}^M \prod_{n=1}^N p(y_n | f_n^{(m)}, \theta) \quad (2.3.9)$$

where $f_n^{(m)} = f^{(m)}(\mathbf{x}_n)$. In our experiments, by being able to evaluate this function for any value of θ , we were able to tune the hyperparameter using a simple grid search.

3. Markov Chain Monte Carlo

3.1 Monte Carlo Methods

As mentioned in Chapter 2, we do not have closed-form expressions for the mean and covariance of the posterior distribution arising from a Gaussian process prior and a Student's t likelihood. The posterior is analytically intractable and can only be expressed up to a normalising constant. Throughout this chapter, for simplicity, the posterior distribution $p(\mathbf{f} \mid \mathbf{y}, \mathbf{X})$ will be written as $p(\mathbf{f})$. We will frequently make use of the unnormalised posterior density $\tilde{p}(\mathbf{f})$ in

$$p(\mathbf{f}) = \frac{1}{Z} \tilde{p}(\mathbf{f}) \quad (3.1.1)$$

where Z is the normalising constant. In our case, $\tilde{p}(\mathbf{f})$ is given by the product of the likelihood and prior, while

$$Z = \int \tilde{p}(\mathbf{f}) d\mathbf{f} \quad (3.1.2)$$

is intractable.

To perform inference, we need to compute expectations under the posterior distribution. Monte Carlo methods provide a general framework for approximating such expectations using samples from the target distribution (MacKay, 2003). Generally, given a function ϕ , if we want to compute $\Phi = E[\phi(\mathbf{f})]$ we can draw M samples $\{\mathbf{f}^{(m)}\}_{m \in [M]}$ from the target distribution, and we can estimate Φ using the following estimator:

$$\bar{\Phi} = \frac{1}{M} \sum_{m=1}^M \phi(\mathbf{f}^{(m)}) \quad (3.1.3)$$

The variance of $\bar{\Phi}$ will be $\text{Var}(\phi)/M$ and will decrease as M increases (MacKay, 2003). Therefore, if we can sample from the posterior, we can approximate posterior expectations and make predictions. Furthermore, by using Monte Carlo methods to sample from the prior, we can evaluate the marginal likelihood by using samples from the prior in Equation (2.3.9), which can be used to tune the hyperparameter of our Gaussian process regression model.

3.1.1 Monte Carlo Sampling

Most sampling methods begin with a source of uniform random numbers. Samples from other distributions can then be obtained by applying transformations that map the uniform distribution to the target distribution. A classical example is the Box-Muller transform (Box and Muller, 1958), which generates samples from a standard Gaussian distribution using uniformly distributed random numbers. Modern numerical libraries such as JAX (Bradbury et al., 2018) provide efficient implementations for sampling from many common distributions. Throughout this work, we use JAX to implement the algorithms presented in this chapter. In addition to its random number generation capabilities, JAX provides automatic differentiation, which plays an important role in the implementation of several of these algorithms, as well as just-in-time (JIT) compilation for program optimisation.

Sampling from the Gaussian process prior is straightforward because the prior is Gaussian. We can generate samples from a standard multivariate Gaussian distribution using JAX and then transform them to have the desired mean and covariance. This is the approach used throughout this work.

Sampling from the posterior is considerably more challenging. Unlike the Gaussian prior, the posterior is sufficiently complex that we cannot construct a simple transformation from a standard distribution to the target distribution. As a result, direct sampling is not feasible. Instead, we turn to Markov chain Monte Carlo (MCMC), a family of methods that produces a sequence of states $(\mathbf{f}^{(m)})_m$ which at some point will behave as though they were samples drawn from the posterior distribution itself. This sequence is generated in such a way that the previous state is used to generate the next. An important advantage of MCMC is that it only requires a function proportional to the target density. In our case, this is precisely the unnormalised posterior density $\tilde{p}(\mathbf{f})$, making MCMC particularly well suited to our problem.

3.1.2 Markov Chain Monte Carlo Methods

Formally, MCMC methods generate samples by constructing a Markov chain whose stationary distribution is the target distribution (Hastings, 1970). A Markov chain is defined by an initial distribution $p^0(\mathbf{f})$ and a transition probability $T(\mathbf{f}^i, \mathbf{f}^j)$, which specifies the probability of moving from state $\mathbf{f}^i$ to state $\mathbf{f}^j$. For the chain to be a valid sampler, the target posterior distribution must be a stationary or invariant distribution of the transition kernel (MacKay, 2003). This requires

$$p(\mathbf{f}^j) = \int T(\mathbf{f}^i, \mathbf{f}^j) p(\mathbf{f}^i) d\mathbf{f}^i \quad (3.1.4)$$

In practice, stationarity is often established by showing that the Markov chain satisfies the detailed balance condition

$$T(\mathbf{f}^i, \mathbf{f}^j) p(\mathbf{f}^i) = T(\mathbf{f}^j, \mathbf{f}^i) p(\mathbf{f}^j) \quad (3.1.5)$$

for all states $\mathbf{f}^i$ and $\mathbf{f}^j$ (MacKay, 2003). Detailed balance implies that the chain is reversible with respect to the target distribution and guarantees that the target distribution is stationary under the transition kernel.

A second requirement is ergodicity. Informally, ergodicity ensures that the chain is able to explore the entire support of the target distribution by making sure that every state is accessible and that it will not become trapped in a subset of the state space (MacKay, 2003). If a chain admits a stationary distribution and is ergodic, the stationary distribution is guaranteed to be unique, and the chain converges to it regardless of its initial state (see, e.g., Robert and Casella, 2004).

As discussed in MacKay (2003), there are some practical considerations when using MCMC. The main issue with MCMC is that until the chain converges, the samples cannot be used to build a reliable estimator. For this reason, it is common to discard an initial portion of the chain, a procedure known as *burn-in*. In addition, successive samples are generally correlated because of the Markov property. As a result, Monte Carlo estimators constructed from MCMC samples typically have a larger variance than those based on independent samples. To address this issue, we might consider only retaining every k -th sample to avoid using consecutive samples in the estimator. However, this means that we will only be able to use a subset of the generated sequence.

The efficiency of an MCMC method therefore depends on how quickly the chain reaches its stationary distribution and how effectively it explores the target distribution. In this work, we are interested in a particular MCMC algorithm which is Riemannian manifold Hamiltonian Monte Carlo (RMHMC) (Girolami and Calderhead, 2011) because it holds the promise of faster convergence. We'll compare it to well-established algorithms like Metropolis-Hastings (MH) (Metropolis et al., 1953; Hastings, 1970) and Hamiltonian Monte Carlo (HMC) (Neal, 2012). In addition, we will introduce Rank-1 RMHMC, a variant of RMHMC that has the potential to be more computationally efficient than RMHMC at the cost of compromising performance.

3.2 Metropolis-Hastings

The Metropolis-Hastings algorithm (Metropolis et al., 1953; Hastings, 1970) is the most basic Markov Chain Monte Carlo method and stands as the foundation of the most elaborate algorithms that we will cover in the next sections. The idea is to construct a Markov chain whose invariant distribution is the target distribution $p(\mathbf{f})$ by repeatedly proposing a new state $\mathbf{f}'$ and accepting or rejecting it according to a probability chosen so that detailed balance is satisfied.

In the formulation in Hastings (1970), proposals are made according to a proposal distribution $q(\mathbf{f} | \mathbf{f}')$ and accepted with probability

$$\alpha(\mathbf{f}, \mathbf{f}') = \min \left(1, \frac{p(\mathbf{f}') q(\mathbf{f} | \mathbf{f}')}{p(\mathbf{f}) q(\mathbf{f}' | \mathbf{f})} \right) \quad (3.2.1)$$

If the proposal is accepted, the next state of the chain becomes $\mathbf{f}'$. Otherwise, the proposal is rejected and the chain remains at $\mathbf{f}$. The transition probability is then defined as

$$T(\mathbf{f}, \mathbf{f}') = q(\mathbf{f} | \mathbf{f}') \alpha(\mathbf{f}, \mathbf{f}') \quad (3.2.2)$$

To show that $p(\mathbf{f})$ is an invariant distribution of this Markov chain, it is sufficient to verify detailed balance. For any pair of states $\mathbf{f}$ and $\mathbf{f}'$,

$$\begin{aligned} p(\mathbf{f}) T(\mathbf{f}, \mathbf{f}') &= p(\mathbf{f}) q(\mathbf{f}' | \mathbf{f}) \min \left(1, \frac{p(\mathbf{f}') q(\mathbf{f} | \mathbf{f}')}{p(\mathbf{f}) q(\mathbf{f}' | \mathbf{f})} \right) \\ &= \min \left(p(\mathbf{f}) q(\mathbf{f}' | \mathbf{f}), p(\mathbf{f}') q(\mathbf{f} | \mathbf{f}') \right) \\ &= p(\mathbf{f}') T(\mathbf{f}', \mathbf{f}) \end{aligned} \quad (3.2.3)$$

Consequently, $p(\mathbf{f})$ satisfies detailed balance and is therefore an invariant distribution of the Markov chain.

Notice that we only need to evaluate the target probability density function when evaluating the acceptance probability. Furthermore, we only need to compute the ratio of probabilities, which can be calculated using the unnormalised density $\tilde{p}(\mathbf{f})$ because the normalising constants will cancel out. As such, the acceptance probability can be written as

$$\alpha(\mathbf{f}, \mathbf{f}') = \min \left(1, \frac{\tilde{p}(\mathbf{f}') q(\mathbf{f} | \mathbf{f}')}{\tilde{p}(\mathbf{f}) q(\mathbf{f}' | \mathbf{f})} \right) \quad (3.2.4)$$

This makes the algorithm highly practical.

Implementation details

For our implementation, we consider the particular case where the proposals are symmetric random walks defined by

$$\mathbf{f}' = \mathbf{f} + \sigma \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (3.2.5)$$

where σ controls the proposed step size.

The proposal distribution being symmetric, we can simplify the acceptance probability to

$$\alpha(\mathbf{f}, \mathbf{f}') = \min \left(1, \frac{\tilde{p}(\mathbf{f}')}{\tilde{p}(\mathbf{f})} \right) = \min(1, \exp(\log \tilde{p}(\mathbf{f}') - \log \tilde{p}(\mathbf{f}))) \quad (3.2.6)$$

Notice that we also rewrote it to use log-probabilities instead of raw density, which is more numerically stable.

We can now combine Equation (3.2.5) to make state proposal and Equation (3.2.6) to decide proposal acceptance, as shown in Algorithm 1, to iteratively generate the Markov chain and eventually obtain samples from the posterior.

The performance of the algorithm is highly dependent on the choice of σ . If σ is too small, most proposals will be accepted, but the chain will explore the parameter space slowly, producing highly correlated samples. Conversely, if σ is too large, most proposals will be rejected and the chain will spend many iterations remaining at the same position. The random-walk behaviour can make exploration inefficient, particularly in high-dimensional cases, motivating the development of more sophisticated methods that we will cover in the next sections.

Algorithm 1 Metropolis-Hastings with Gaussian Proposal

Require:

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}_{\text{init}}$: initial state
- 3: t_{max} : number of iterations
- 4: σ : standard deviation of the Gaussian proposal

Returns

- 5: $\bar{r}$: rejection rate
- 6: $\{\mathbf{f}^{(t)}\}_{t=1}^{t_{\text{max}}}$: MCMC samples

Initialise

- 7: $\mathbf{f}^{(1)} \leftarrow \mathbf{f}_{\text{init}}$, rejections $\leftarrow 0$

Begin

- 8: **for** $t = 1$ to t_{max} **do**
- 9: Sample $\boldsymbol{\xi} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; $\mathbf{f}' \leftarrow \mathbf{f}^{(t)} + \sigma \boldsymbol{\xi}$ $\triangleright$ Symmetric Gaussian random-walk proposal (3.2.5)
- 10: Sample $u \sim \text{Uniform}(0, 1)$
- 11: **if** $u \leq \exp(\log \tilde{p}(\mathbf{f}') - \log \tilde{p}(\mathbf{f}^{(t)}))$ **then** $\triangleright$ Accept proposal (3.2.6)
- 12: $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}'$
- 13: **else** $\triangleright$ Reject proposal
- 14: rejections $\leftarrow$ rejections + 1
- 15: $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}^{(t)}$
- 16: **end if**
- 17: **end for**
- 18: $\bar{r} \leftarrow \text{rejections} / t_{\text{max}}$
- 19: **return** $\bar{r}$, $\{\mathbf{f}^{(t)}\}$

End

3.3 Hamiltonian Monte Carlo

Hamiltonian Monte Carlo (HMC) samples from a distribution by making use of Hamiltonian dynamics (Neal, 2012). This method improves on standard Metropolis-Hastings by making proposals after exploring the parameter space using paths simulated with Hamiltonian dynamics instead of random walk. This enables the sampler to make much larger moves across the parameter space, potentially reducing correlation between successive samples.

In Hamiltonian dynamics, the energy of the system is described as a function of an N -dimensional position vector, which will correspond to our variable $\mathbf{f}$, and an N -dimensional momentum vector $\mathbf{p}$, which is known as the Hamiltonian. The evolution of $\mathbf{f}$ and $\mathbf{p}$ in time is described through the Hamiltonian equations of motion

$$\begin{aligned}\frac{d\mathbf{f}}{ds} &= \frac{\partial H}{\partial \mathbf{p}} \\ \frac{d\mathbf{p}}{ds} &= -\frac{\partial H}{\partial \mathbf{f}}\end{aligned}\tag{3.3.1}$$

Sampling from a distribution using HMC will require us to use those equations in order to simulate the time evolution of a Hamiltonian system from the current sample to our next proposal.

Additionally, we must consider the following properties of the Hamiltonian:

Energy conservation: The Hamiltonian $H(\mathbf{f}, \mathbf{p})$ remains constant during time evolution.

Volume preservation: The volume of a region of $(\mathbf{f}, \mathbf{p})$ states remains invariant during time evolution.

Time reversibility: The dynamics are invariant under time reversal. Evolving forward, negating the momentum ($\mathbf{p} \rightarrow -\mathbf{p}$), and evolving forward for the same duration returns the system to its initial state.

We will see that HMC takes these properties into account when sampling from a distribution.

In order to sample a variable $\mathbf{f} \in \mathbb{R}^N$ from our posterior distribution using HMC, we introduce a Gaussian random variable $\mathbf{p} \in \mathbb{R}^N$ with $\mathbf{p} \sim \mathcal{N}(0, \mathbf{M})$ that acts as a momentum variable in the Hamiltonian. We can then write the joint distribution as follows:

$$p(\mathbf{f}, \mathbf{p}) \propto \exp(-H(\mathbf{f}, \mathbf{p}))\tag{3.3.2}$$

where $H(\mathbf{f}, \mathbf{p})$ is the Hamiltonian. Given the independence of $\mathbf{f}$ and $\mathbf{p}$, it follows that:

$$H(\mathbf{f}, \mathbf{p}) = -\log \tilde{p}(\mathbf{f}) + \frac{1}{2} \mathbf{p}^\top \mathbf{M}^{-1} \mathbf{p}\tag{3.3.3}$$

The matrix $\mathbf{M}$ represents the mass matrix, which acts as a preconditioner and can be used to inform the sampler about the scales and correlation of the variables (Neal, 2012). As this matrix defines a covariance matrix and needs to be invertible, it is required to be symmetric and positive-definite. Following the terminology of Golub and Van Loan (2013), a symmetric matrix $\mathbf{A}$ is said to be positive-definite if for any vector $\mathbf{v}$, $\mathbf{v}^\top \mathbf{A} \mathbf{v} > 0$; if $\mathbf{v}^\top \mathbf{A} \mathbf{v} \geq 0$ it is positive semi-definite. In most of our experiments, this mass matrix would be set as the identity matrix, which prevents us from having to deal with an expensive matrix inverse operation.

In the t -th iteration of the algorithm with current state $(\mathbf{f}^{(t)}, \mathbf{p}^{(t)})$, to get a new proposal $\mathbf{f}'$, we first sample a momentum $\mathbf{p}^{(t+1)}$ from the Gaussian distribution. Using this momentum, we simulate a path to the next proposal using a numerical integrator on the Hamiltonian equations of motion (3.3.1). The new proposal $(\mathbf{f}', \mathbf{p}')$ is then accepted with probability

$$\alpha((\mathbf{f}^{(t)}, \mathbf{p}^{(t)}), (\mathbf{f}', \mathbf{p}')) = \min \left(1, \frac{p(\mathbf{f}', \mathbf{p}')}{p(\mathbf{f}^{(t)}, \mathbf{p}^{(t)})} \right) = \min \left(1, \exp(H(\mathbf{f}', \mathbf{p}') - H(\mathbf{f}^{(t)}, \mathbf{p}^{(t)})) \right)\tag{3.3.4}$$

This is the Metropolis-Hastings acceptance probability (3.2.6), which in some ways makes us think of HMC as a Metropolis-Hastings algorithm on the augmented state space $(\mathbf{f}, \mathbf{p})$ with more sophisticated proposals.

In order to obtain a valid Markov Chain Monte Carlo sampler, following the properties of the Hamiltonian, this integrator must be time-reversible and volume-preserving. Those properties are essential as they ensure a symmetric proposal distribution, and along with α as acceptance probability, HMC is ensured to satisfy detailed balance with respect to the joint distribution $p(\mathbf{f}, \mathbf{p})$ (Neal, 2012). We will also notice that if the integrator were also to satisfy energy conservation, which is hard to satisfy due to the limitation of having to solve the integrals using discrete time, every proposal will always be accepted since we will have $\alpha = 1$.

In practice, the Hamiltonian dynamic is simulated using the *leapfrog integrator*, which is both reversible and volume-preserving. The *leapfrog integrator* advances the system by first performing a half-step update of the momentum, followed by a full-step update of the position, and concluding with a second half-step update of the momentum:

$$\mathbf{p}(s + \frac{\epsilon}{2}) = \mathbf{p}(s) - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}(s), \mathbf{p}(s)) \quad (3.3.5)$$

$$\mathbf{f}(s + \epsilon) = \mathbf{f}(s) + \epsilon \nabla_{\mathbf{p}} H(\mathbf{f}(s), \mathbf{p}(s + \frac{\epsilon}{2})) \quad (3.3.6)$$

$$\mathbf{p}(s + \epsilon) = \mathbf{p}(s + \frac{\epsilon}{2}) - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}(s + \epsilon), \mathbf{p}(s + \frac{\epsilon}{2})) \quad (3.3.7)$$

where ϵ is the step size. This is repeated for τ steps to produce the proposal $(\mathbf{f}', \mathbf{p}')$. The gradients of the Hamiltonian can be derived and written as follows:

$$\nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p}) = -\nabla \log \tilde{p}(\mathbf{f}) \quad (3.3.8)$$

$$\nabla_{\mathbf{p}} H(\mathbf{f}, \mathbf{p}) = \mathbf{M}^{-1} \mathbf{p} \quad (3.3.9)$$

Implementation details

Algorithm 2 summarises the complete HMC procedure. At each iteration, a momentum variable is first sampled from its Gaussian distribution and combined with the current position to form the initial state. The corresponding Hamiltonian is evaluated using Equation (3.3.3). A proposal is then generated by simulating Hamiltonian dynamics using the leapfrog integrator described in Algorithm 3. Once the proposal is obtained, its Hamiltonian is computed, and it is accepted or rejected according to the Metropolis-Hastings acceptance probability given in Equation (3.3.4). If the proposal is accepted, the chain moves to the new position; otherwise, the current position is retained. The proposal generation step as depicted in Algorithm 3 generates a new proposal by simulating the Hamiltonian for τ steps using Equations (3.3.5), (3.3.6) and (3.3.7)

To sample a momentum $\mathbf{p}$, we first sample $\mathbf{z}$ from a standard Gaussian using JAX and then multiply it by a matrix $\mathbf{B}$ such that $\mathbf{B}\mathbf{B}^\top = \mathbf{M}$. The resulting random variable will still be Gaussian, and we can easily check that by computing the covariance of $\mathbf{B}\mathbf{z}$, we will have

$$E[(\mathbf{B}\mathbf{z})(\mathbf{B}\mathbf{z})^\top] = \mathbf{B}E[\mathbf{z}\mathbf{z}^\top]\mathbf{B}^\top = \mathbf{B}\mathbf{B}^\top = \mathbf{M} \quad (3.3.10)$$

In practice, the mass matrix would be required to be positive definite, and this matrix can be obtained by taking the Cholesky decomposition

$$\mathbf{L}\mathbf{L}^\top = \mathbf{M} + \beta \mathbf{I} \quad (3.3.11)$$

where β is a small jitter term for numerical stability. We can then obtain the sample by computing $\mathbf{p} = \mathbf{L}\mathbf{z}$, as seen in Algorithm 2.

The Hamiltonian contains the quadratic form $\mathbf{p}^\top \mathbf{M}^{-1} \mathbf{p}$, which requires evaluating $\mathbf{M}^{-1} \mathbf{p}$. We also need to do this operation when simulating the Hamiltonian dynamic as we can see in the expression of

the gradient (3.3.9). However, inverting the covariance matrix is computationally expensive. It is more efficient to compute the LU decomposition, or the Cholesky decomposition in the case of a positive definite matrix, and then solve the system

$$\mathbf{L}\mathbf{L}^\top \mathbf{w} = \mathbf{p} \quad (3.3.12)$$

which gives us $\mathbf{w} = \mathbf{M}^{-1}\mathbf{p}$ without explicitly computing the inverse (see, e.g., [Golub and Van Loan, 2013](#)). In our implementation, as seen in both Algorithm 2 and Algorithm 3, the Cholesky factor is computed at initialisation and then used throughout the algorithm for sampling, computing the Hamiltonian and simulating the dynamic.

Solving those linear systems is the most expensive operation of the algorithms. When this matrix is chosen to be diagonal, every operation will be significantly faster because sampling requires only computing the square root of the diagonal entries, whereas inverting the matrix only requires inverting the diagonal. However, in our implementation, we did not account for those optimisations for diagonal matrices, except when the mass matrix is chosen to be the identity, as was the case in the majority of our experiments.

In order to compute the gradient of the log-probability density needed in Algorithm 3, we use automatic differentiation of JAX. As reviewed in [Baydin et al. \(2018\)](#), automatic differentiation techniques work by tracing every elementary operation with known derivatives applied to the input, computing their derivatives, and finally combining them. The derivatives of each operation are generally computed and combined in one of two ways: forward mode, which computes and combines the derivatives as the operation is applied, and backward mode, which computes and combines the derivatives using the chain rule from the output back to the input.

Algorithm 2 Hamiltonian Monte Carlo**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}_{\text{init}}$: initial position
- 3: t_{max} : number of iterations
- 4: ϵ : leapfrog step size
- 5: τ : number of leapfrog steps
- 6: $\mathbf{M}$: mass matrix (covariance of momentum)
- 7: β : Jitter term

Returns

- 8: $\bar{r}$: rejection rate
- 9: $\{\mathbf{f}^{(t)}\}_{t=1}^{t_{\text{max}}}$: MCMC samples

Initialise

- 10: $\mathbf{f}^{(1)} \leftarrow \mathbf{f}_{\text{init}}$, rejections $\leftarrow 0$
- 11: $\mathbf{L} \leftarrow \text{Cholesky}(\mathbf{M} + \beta \mathbf{I})$

▷ Cholesky decomposition (3.3.11)

Begin

- 12: **for** $t = 1$ to t_{max} **do**
- 13: Sample $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$; $\mathbf{p} \leftarrow \mathbf{L}\mathbf{z}$
- 14: $\mathbf{w} \leftarrow \text{CholeskySolve}(\mathbf{L}, \mathbf{p})$
- 15: $\mathbf{H} \leftarrow \frac{1}{2}\mathbf{p}^\top \mathbf{w} - \log \tilde{p}(\mathbf{f}^{(t)})$
- 16: $(\mathbf{f}', \mathbf{p}') \leftarrow \text{Leapfrog}(\mathbf{f}^{(t)}, \mathbf{p}, \mathbf{L}, \epsilon, \tau, \log \tilde{p})$
- 17: $\mathbf{w}' \leftarrow \text{CholeskySolve}(\mathbf{L}, \mathbf{p}')$
- 18: $\mathbf{H}' \leftarrow \frac{1}{2}\mathbf{p}'^\top \mathbf{w}' - \log \tilde{p}(\mathbf{f}')$
- 19: Sample $u \sim \text{Uniform}(0, 1)$
- 20: **if** $u < \exp(\mathbf{H} - \mathbf{H}')$ **then**
- 21: $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}'$
- 22: **else**
- 23: rejections $\leftarrow$ rejections + 1
- 24: $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}^{(t)}$
- 25: **end if**
- 26: **end for**
- 27: $\bar{r} \leftarrow \text{rejections} / t_{\text{max}}$
- 28: **return** $\bar{r}, \{\mathbf{f}^{(t)}\}$

▷ $\mathbf{p} \sim \mathcal{N}(0, \mathbf{M})$ ▷ $\mathbf{w} = \mathbf{M}^{-1}\mathbf{p}$ (3.3.12)

▷ Initial Hamiltonian (3.3.3)

▷ New proposal (Algorithm 3)

▷ $\mathbf{w}' = \mathbf{M}^{-1}\mathbf{p}'$ (3.3.12)

▷ Proposal Hamiltonian (3.3.3)

▷ Accept proposal (3.3.4)

▷ Reject proposal

End

Algorithm 3 Leapfrog Integrator Leapfrog**Require:**

- 1: $\mathbf{f}$: current position
- 2: $\mathbf{p}$: current momentum
- 3: $\mathbf{L}$: cholesky factor of the mass matrix
- 4: ϵ : step size
- 5: τ : number of steps
- 6: $\log \tilde{p}$: unnormalised log-probability density

Returns

- 7: $(\mathbf{f}, \mathbf{p})$: new state proposal
Define $\nabla(-\log \tilde{p})(\mathbf{f}) = -\frac{d \log \tilde{p}}{d\mathbf{f}}(\mathbf{f})$ computed via Autodiff

Begin

- 8: **for** $t = 1$ to τ **do**
- 9: $\mathbf{p} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla(-\log \tilde{p})(\mathbf{f})$ ▷ Half step momentum (3.3.5)
- 10: $\mathbf{w} \leftarrow \text{CholeskySolve}(\mathbf{L}, \mathbf{p})$ ▷ $\mathbf{w} = \mathbf{M}^{-1}\mathbf{p}$ (3.3.12)
- 11: $\mathbf{f} \leftarrow \mathbf{f} + \epsilon \mathbf{w}$ ▷ Full step position (3.3.6)
- 12: $\mathbf{p} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla(-\log \tilde{p})(\mathbf{f})$ ▷ Half step momentum (3.3.7)
- 13: **end for**
- 14: **return** $(\mathbf{f}, \mathbf{p})$

End

3.4 Riemannian manifold Hamiltonian Monte Carlo

As argued in the previous section, Hamiltonian Monte Carlo is a significant improvement over Metropolis-Hastings. However, tuning the mass matrix $\mathbf{M}$ is a difficult task. If the distribution of $\mathbf{f}$ was roughly Gaussian and we had an estimate of its covariance matrix, we could set the mass matrix as the inverse of the covariance matrix to improve HMC by compensating for the difference in scale as well as the correlation between the variables (Neal, 2012). In practice, when sampling from a complex distribution, we do not necessarily know the shape of the distribution or its covariance matrix. Riemannian manifold Hamiltonian Monte Carlo (RMHMC) (Girolami and Calderhead, 2011) addresses this issue by having the covariance of the momentum depend on $\mathbf{f}$ and using it to incorporate local geometry into the sampler via a metric tensor. This metric tensor is a position-dependent, symmetric, positive-definite matrix that can be used to define a dot product.

In Girolami and Calderhead (2011), the authors motivate the use of Riemannian geometry by introducing the Fisher-Rao metric, which establishes a notion of distance between parametric distributions. They argue that such a metric can be adapted to HMC to significantly improve sampling efficiency. Among the discussed choices, they mentioned the observed Fisher matrix as a reasonable metric, which corresponds to the Hessian of the distribution's negative log-probability function even though it is not guaranteed to be positive definite. In our experiment we were able to sample from the posterior using this metric.

To sample from our posterior distribution in the Riemannian manifold Hamiltonian Monte Carlo algorithms, we define the momentum random vector $\mathbf{p}$ as a Gaussian with covariance matrix $\mathbf{G}(\mathbf{f})$, where $\mathbf{G}(\mathbf{f})$ is the metric tensor capturing the local geometry of the distribution space. As opposed to HMC, the momentum will be dependent on $\mathbf{f}$. Therefore, the Hamiltonian will be expressed as follows:

$$H(\mathbf{f}, \mathbf{p}) = -\log \tilde{p}(\mathbf{f}) + \frac{1}{2} \log \det \mathbf{G}(\mathbf{f}) + \frac{1}{2} \mathbf{p}^\top \mathbf{G}(\mathbf{f})^{-1} \mathbf{p} \quad (3.4.1)$$

In HMC, the Hamiltonian was separable, which meant that potential and kinetic energy were only functions of $\mathbf{f}$ and $\mathbf{p}$, respectively. In RMHMC, this is no longer the case because the kinetic energy component of the Hamiltonian now depends on both vectors. As a result, in [Girolami and Calderhead \(2011\)](#), the authors stated that the *leapfrog integrator* is no longer valid, and we would need another integrator that satisfies both time reversibility and volume preservation. They proposed the *generalised leapfrog integrator* as a solution. Similar to the *leapfrog integrator*, it performs a half-step momentum, followed by a full-step position, and finally, a half-step momentum. However, unlike the former, the first two steps are defined implicitly and must be solved using fixed-point iterations.

$$\mathbf{p}(s + \frac{\epsilon}{2}) = \mathbf{p}(s) - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}(s), \mathbf{p}(s + \frac{\epsilon}{2})) \quad (3.4.2)$$

$$\mathbf{f}(s + \epsilon) = \mathbf{f}(s) + \frac{\epsilon}{2} [\nabla_{\mathbf{p}} H(\mathbf{f}(s), \mathbf{p}(s + \frac{\epsilon}{2})) + \nabla_{\mathbf{p}} H(\mathbf{f}(s + \epsilon), \mathbf{p}(s + \frac{\epsilon}{2}))] \quad (3.4.3)$$

$$\mathbf{p}(s + \epsilon) = \mathbf{p}(s + \frac{\epsilon}{2}) - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}(s + \epsilon), \mathbf{p}(s + \frac{\epsilon}{2})) \quad (3.4.4)$$

Implementation details

Algorithm 4 summarises the whole RMHMC process. It has a similar structure to Algorithm 2 except that the Cholesky decomposition can no longer be computed at initialisation, as the metric is position-dependent. The Hamiltonian also differs, as it now follows Equation (3.4.1) as implemented in Algorithm 6 and the proposals are now generated using the generalised leapfrog integrator which is described in Algorithm 5 following the Equations (3.4.2), (3.4.3) and (3.4.4). The accept-reject criteria remain the same for both RMHMC and HMC, as both use Equation (3.3.4)

Equations (3.4.2) and (3.4.3) are implicit, and at any step s of the leapfrog integrator, we can derive the fixed-point equations:

$$\mathbf{p}_{k+1} = \mathbf{p}(s) - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}(s), \mathbf{p}_k) \quad (3.4.5)$$

$$\mathbf{f}_{k+1} = \mathbf{f}(s) + \frac{\epsilon}{2} [\nabla_{\mathbf{p}} H(\mathbf{f}(s), \mathbf{p}(s + \frac{\epsilon}{2})) + \nabla_{\mathbf{p}} H(\mathbf{f}_k, \mathbf{p}(s + \frac{\epsilon}{2}))] \quad (3.4.6)$$

At each step s , the first half-step momentum update is performed to compute $\mathbf{p}(s + \frac{\epsilon}{2})$. Starting from $\mathbf{p}_0 = \mathbf{p}(s)$, we perform κ fixed-point iterations according to Equation (3.4.5) and set $\mathbf{p}(s + \frac{\epsilon}{2}) = \mathbf{p}_\kappa$. Next, the full-step position update is used to compute $\mathbf{f}(s)$. Beginning with $\mathbf{f}_0 = \mathbf{f}(s)$, we apply another κ fixed-point iterations using Equation (3.4.6), yielding $\mathbf{f}(s) = \mathbf{f}_\kappa$. The overall procedure is summarised in Algorithm 5.

Notice that taking the derivative of the Hamiltonian with respect to $\mathbf{p}$ gives us $\nabla_{\mathbf{p}} H(\mathbf{f}, \mathbf{p}) = \mathbf{G}(\mathbf{f})^{-1} \mathbf{p}$. We would need to compute this operation throughout the algorithm, not only to compute the gradient of the Hamiltonian with respect to $\mathbf{p}$, but also to compute the Hamiltonian itself as seen in Equation (3.4.1). Similar to our HMC implementation, we first compute the Cholesky decomposition

$$\mathbf{L}_f \mathbf{L}_f^\top = \mathbf{G}(\mathbf{f}) + \beta \mathbf{I} \quad (3.4.7)$$

where β is a jitter term, and then solve the system

$$\mathbf{L}_f \mathbf{L}_f^\top \mathbf{w} = \mathbf{p} \quad (3.4.8)$$

which gives us $\mathbf{w} = \mathbf{G}(\mathbf{f})^{-1} \mathbf{p}$. We will also use this Cholesky factor to sample $\mathbf{p}$ by first sampling $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ and then computing $\mathbf{p} = \mathbf{L}_f \mathbf{z}$.

Furthermore, we can efficiently compute the determinant of the metric tensor by multiplying the diagonal terms of the Cholesky factor and then squaring it.

$$\det \mathbf{G}(\mathbf{f}) = \left(\prod_{n=1}^N [\mathbf{L}_f]_{nn} \right)^2 \quad (3.4.9)$$

Consequently, we can compute the log-determinant as follows:

$$\log \det \mathbf{G}(\mathbf{f}) = 2 \sum_{n=1}^N \log[\mathbf{L}_{\mathbf{f}}]_{nn} \quad (3.4.10)$$

Since the Cholesky has to be computed anyway, Algorithm 6 uses Equation (3.4.10) to make computing the Hamiltonian more efficient.

We can also derive the gradient of the Hamiltonian with respect to $\mathbf{f}$ and use this expression to compute it,

$$[\nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p})]_n = -\frac{\partial \log \tilde{\mathbf{p}}(\mathbf{f})}{\partial f_n} + \frac{1}{2} \text{tr} \left(\mathbf{G}(\mathbf{f})^{-1} \frac{\partial \mathbf{G}(\mathbf{f})}{\partial f_n} \right) - \frac{1}{2} \mathbf{p}^\top \mathbf{G}(\mathbf{f})^{-1} \frac{\partial \mathbf{G}(\mathbf{f})}{\partial f_n} \mathbf{G}(\mathbf{f})^{-1} \mathbf{p} \quad (3.4.11)$$

As we can see, we will need to compute the derivative of the metric tensor, which will result in an order-3 tensor of dimension $N \times N \times N$. Additionally, we will also need to solve multiple linear systems in order to compute every operation involving the inverse of the metric tensor. This will be computationally expensive, and instead we will make use of JAX automatic differentiation in Algorithm 5 to compute this gradient, which gives us a faster implementation.

The metric tensor, which in this case is the Hessian, was also calculated using JAX automatic differentiation.

Algorithm 4 Riemannian Manifold Hamiltonian Monte Carlo**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}_{\text{init}}$: initial position
- 3: t_{max} : number of iterations
- 4: ϵ : leapfrog step size
- 5: τ : number of generalized leapfrog steps
- 6: κ : fixed-point iterations per leapfrog step
- 7: β : Jitter term

Returns

- 8: $\bar{r}$: rejection rate
- 9: $\{\mathbf{f}^{(t)}\}_{t=1}^{t_{\text{max}}}$: MCMC samples

Initialise

10: $\mathbf{f}^{(1)} \leftarrow \mathbf{f}_{\text{init}}, \quad \text{rejections} \leftarrow 0$

Begin

```

11: for  $t = 1$  to  $t_{\text{max}}$  do
12:    $\mathbf{G} \leftarrow \nabla^2(-\log \tilde{p})(\mathbf{f}^{(t)})$  ▷ Metric tensor (Hessian) via Autodiff
13:    $\mathbf{L} \leftarrow \text{Cholesky}(\mathbf{G} + \beta \mathbf{I})$  ▷ Equation (3.4.7)
14:   Sample  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}); \quad \mathbf{p} \leftarrow \mathbf{L}\mathbf{z}$  ▷  $\mathbf{p} \sim \mathcal{N}(\mathbf{0}, \mathbf{G}(\mathbf{f}^{(t)}))$ 
15:    $H \leftarrow \text{Hamiltonian}(\mathbf{f}^{(t)}, \mathbf{p}, \beta, \log \tilde{p})$  ▷ Initial Hamiltonian (Algorithm 6)
16:    $(\mathbf{f}', \mathbf{p}') \leftarrow \text{GenLeapfrog}(\log \tilde{p}, \mathbf{f}^{(t)}, \mathbf{p}, \epsilon, \tau, \kappa, \beta)$  ▷ New proposal (Algorithm 5)
17:    $H' \leftarrow \text{Hamiltonian}(\mathbf{f}', \mathbf{p}', \beta, \log \tilde{p})$  ▷ Proposal Hamiltonian (Algorithm 6)
18:   Sample  $u \sim \text{Uniform}(0, 1)$ 
19:   if  $u < \exp(H - H')$  then ▷ Accept proposal (3.3.4)
20:      $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}'$ 
21:   else ▷ Reject proposal
22:     rejections  $\leftarrow$  rejections + 1
23:      $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}^{(t)}$ 
24:   end if
25: end for
26:  $\bar{r} \leftarrow \text{rejections} / t_{\text{max}}$ 
27: return  $\bar{r}, \{\mathbf{f}^{(t)}\}$ 

```

End

Algorithm 5 Generalized Leapfrog Integrator GenLeapfrog**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}, \mathbf{p}$: current position and momentum
- 3: ϵ : step size
- 4: τ : number of steps
- 5: κ : fixed-point iterations
- 6: β : Jitter term

Returns

- 7: $(\mathbf{f}, \mathbf{p})$: updated state

Define, $\nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p}) := \frac{\partial}{\partial \mathbf{f}} \text{Hamiltonian}(\mathbf{f}, \mathbf{p}, \beta, \log \tilde{p})$ computed via *Autodiff*

Begin

```

8: for  $t = 1$  to  $\tau$  do
9:    $\tilde{\mathbf{p}} \leftarrow \mathbf{p}$ 
10:  for  $k = 1$  to  $\kappa$  do                                ▷ Implicit half-step for momentum (3.4.2)
11:     $\tilde{\mathbf{p}} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}, \tilde{\mathbf{p}})$ 
12:  end for
13:   $\mathbf{p} \leftarrow \tilde{\mathbf{p}}$ 
14:   $\mathbf{G} \leftarrow \nabla^2(-\log \tilde{p})(\mathbf{f})$                     ▷ Metric (Hessian) at current  $\mathbf{f}$  via Autodiff
15:   $\mathbf{L}_{\mathbf{f}} \leftarrow \text{Cholesky}(\mathbf{G} + \beta \mathbf{I})$               ▷ Equation (3.4.7)
16:   $\mathbf{w} \leftarrow \text{CholeskySolve}(\mathbf{L}_{\mathbf{f}}, \mathbf{p})$               ▷  $\mathbf{w} = \mathbf{G}(\mathbf{f})^{-1} \mathbf{p}$  (3.4.8)
17:   $\tilde{\mathbf{f}} \leftarrow \mathbf{f}, \tilde{\mathbf{w}} \leftarrow \mathbf{w}$ 
18:  for  $k = 1$  to  $\kappa$  do                                ▷ Implicit full step for position (3.4.3)
19:     $\tilde{\mathbf{f}} \leftarrow \mathbf{f} + \frac{\epsilon}{2}(\mathbf{w} + \tilde{\mathbf{w}})$ 
20:     $\tilde{\mathbf{G}} \leftarrow \nabla^2(-\log \tilde{p})(\tilde{\mathbf{f}})$                 ▷ Metric (Hessian) at updated  $\mathbf{f}$  via Autodiff
21:     $\tilde{\mathbf{L}}_{\mathbf{f}} \leftarrow \text{Cholesky}(\tilde{\mathbf{G}} + \beta \mathbf{I})$           ▷ Equation (3.4.7)
22:     $\tilde{\mathbf{w}} \leftarrow \text{CholeskySolve}(\tilde{\mathbf{L}}_{\mathbf{f}}, \mathbf{p})$           ▷  $\tilde{\mathbf{w}} = \mathbf{G}(\tilde{\mathbf{f}})^{-1} \mathbf{p}$  (3.4.8)
23:  end for
24:   $\mathbf{f} \leftarrow \tilde{\mathbf{f}}$ 
25:   $\mathbf{p} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p})$           ▷ Explicit half-step for momentum (3.4.4)
26: end for
27: return  $(\mathbf{f}, \mathbf{p})$ 

```

End

Algorithm 6 Hamiltonian Hamiltonian**Require:**

- 1: $\mathbf{f}, \mathbf{p}$: position and momentum
- 2: β : Jitter term
- 3: $\log \tilde{p}$: unnormalised log-probability density

Returns

- 4: H : Hamiltonian value $H(\mathbf{f}, \mathbf{p}) = \frac{1}{2}\mathbf{p}^\top \mathbf{G}(\mathbf{f})^{-1}\mathbf{p} + \frac{1}{2}\log \det \mathbf{G}(\mathbf{f}) - \log \tilde{p}(\mathbf{f})$

Begin

- 5: $\mathbf{G} \leftarrow \nabla^2(-\log \tilde{p})(\mathbf{f})$ ▷ Hessian via Autodiff
- 6: $\mathbf{L}_f \leftarrow \text{Cholesky}(\mathbf{G} + \beta \mathbf{I})$ ▷ Equation (3.4.7)
- 7: $\mathbf{w} \leftarrow \text{CholeskySolve}(\mathbf{L}_f, \mathbf{p})$ ▷ $\mathbf{w} = \mathbf{G}(\mathbf{f})^{-1}\mathbf{p}$ (3.4.8)
- 8: $H \leftarrow \frac{1}{2}\mathbf{p}^\top \mathbf{w} + \sum_n \log[\mathbf{L}_f]_{nn} - \log \tilde{p}(\mathbf{f})$ ▷ Equations (3.4.10) and (3.4.1)
- 9: **return** H

End

3.5 Rank-1 RMHMC

The main limitation of RMHMC is its computational cost. As discussed in [Girolami and Calderhead \(2011\)](#), the main computational bottleneck of the algorithms arises from repeatedly solving linear systems involving the metric tensor. In the worst case, each of these operations scales as $O(N^3)$, and they must be performed repeatedly throughout the simulation. While standard HMC can also have a worst-case cost of $O(N^3)$ when a dense mass matrix is used, this mass matrix is fixed and does not need to be recomputed during sampling, which in practice makes HMC less expensive than RMHMC.

To reduce the computational cost of RMHMC, we propose using a low-rank approximation of the Hessian, focusing on the rank-1 case. As we will see, this can make each sampling step much cheaper to compute. Although low-rank approximations are often used to speed up computations in many areas, they have not been thoroughly studied in the context of RMHMC, making this approach an entirely exploratory and novel work.

The main idea comes from the role of the metric tensor, which describes the local geometry of the target distribution. By using a rank-1 approximation, we keep only the most important direction of curvature and ignore the rest. In other words, the sampler still uses some geometric information, but at a much lower computational cost than full RMHMC.

We believe that this approach can provide a useful middle ground between standard HMC, which uses the identity matrix as a mass matrix, and RMHMC. Standard HMC does not take local geometry into account, while RMHMC uses the full geometric information available. A low-rank approximation allows us to incorporate part of this information while avoiding much of the extra computational cost. Ideally, it should be able to explore the space of the distribution better than standard HMC while running at a fraction of the cost of RMHMC.

In this work, we focus on the rank-1 case because it offers the largest potential reduction in cost allowing us to reduce the cost of solving each linear system to $O(N)$. It also provides a simple way to assess how much benefit can be obtained from using only the most important geometric information. As such, it serves as a lower bound for understanding the performance of low-rank approximations in RMHMC.

Now that we have established the motivation for this exploratory work, we must figure out how to find the rank-1 approximation of the tensor metric. We begin by reviewing the foundational results

leading to our method of computing this rank-1 approximation, taking [Golub and Van Loan \(2013\)](#) as a primary general reference. The notion of low-rank approximation is tightly related to Singular Value Decomposition (SVD), which is stated as follows:

3.5.1 Theorem (Singular Value Decomposition). *If $\mathbf{A} \in \mathbb{R}^{M \times N}$, then there exist orthogonal matrices*

$$\mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_M) \in \mathbb{R}^{M \times M} \quad \text{and} \quad \mathbf{V} = (\mathbf{v}_1, \dots, \mathbf{v}_N) \in \mathbb{R}^{N \times N} \quad (3.5.1)$$

such that,

$$\mathbf{U}^\top \mathbf{A} \mathbf{V} = \Sigma = \text{diag}(\sigma_1, \dots, \sigma_P) \in \mathbb{R}^{M \times N} \quad (3.5.2)$$

where $P = \min\{M, N\}$, and $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_P \geq 0$. σ_i are called singular values, $\mathbf{u}_i$ are left singular vectors, and $\mathbf{v}_i$ are right singular vectors.

In theory, the metric tensor is symmetric and positive-definite. We will see that for those matrices, the SVD can take a particular form. Symmetric and positive-definite matrices possess eigenvalues that are strictly positive. Moreover, for a symmetric matrix we have the following result:

3.5.2 Theorem (Symmetric Schur Decomposition). *If $\mathbf{A} \in \mathbb{R}^{N \times N}$ is symmetric, then there exists a real orthogonal $\mathbf{Q}$ such that*

$$\mathbf{Q}^\top \mathbf{A} \mathbf{Q} = \Lambda = \text{diag}(\lambda_1, \dots, \lambda_N) \quad (3.5.3)$$

where λ_n are eigenvalues of $\mathbf{A}$ with $\mathbf{Q}_{:,n}$ being their corresponding normalised eigenvectors.

As a consequence, if $\mathbf{A}$ is a symmetric and positive-definite matrix, if the eigenvalues were arranged from largest to smallest, we would get the SVD decomposition of $\mathbf{A}$ with $\mathbf{U} = \mathbf{V} = \mathbf{Q}$.

We can now formally characterise the best low-rank approximation of a given matrix. This characterisation follows from a classical result introduced in [Eckart and Young \(1936\)](#). This result can be stated as follows:

3.5.3 Theorem (Eckart-Young-Mirsky theorem). *If $k < r = \text{rank}(\mathbf{A})$ and $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ be the truncated SVD of $\mathbf{A}$ obtained by truncating Σ to only keep the top- k singular values, then*

$$\min_{\text{rank}(\mathbf{B})=k} \|\mathbf{A} - \mathbf{B}\|_2 = \|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1} \quad (3.5.4)$$

where $\|\cdot\|_2$ denotes the matrix norm $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$ with $\lambda_{\max}(\mathbf{A}^\top \mathbf{A})$ being the largest eigenvalue in absolute value of $\mathbf{A}^\top \mathbf{A}$.

Based on the theorem stated above, the best low-rank approximation of the metric tensor is obtained via its truncated SVD. In particular, the best rank-1 approximation is given by the largest singular value together with its associated left and right singular vectors. For symmetric positive-definite matrices, this singular value coincides with the largest eigenvalue, and both singular vectors coincide with the corresponding normalised eigenvector. Consequently, the resulting rank-1 approximation is itself symmetric.

A symmetric rank-1 matrix can be specified entirely using a single vector. For our use case, the rank-1 approximation of the metric tensor $\mathbf{G}(\mathbf{f})$ is entirely specified by a vector $\mathbf{u}(\mathbf{f}) = \sqrt{\sigma_1(\mathbf{f})} \mathbf{v}_1(\mathbf{f})$, where $\sigma_1(\mathbf{f})$ is its largest singular value and $\mathbf{v}_1(\mathbf{f})$ the corresponding singular vector.

To compute this vector, we propose two different approaches. The first is to compute the metric tensor and then compute its SVD as seen in Algorithm 10. This will give us all the singular values and singular vectors of our metric tensor, and we will keep only the leading singular value with its corresponding

singular vector. Alternatively, as we should assume the metric tensor to be positive definite, we can use power iteration to compute the leading eigenvalue with its eigenvector, giving us our rank-1 metric approximation. This is described in Algorithm 11. Furthermore, due to the fact that we are using the Hessian as our metric tensor, we can make this method more memory efficient. As shown in Pearlmutter (1994), by using automatic differentiation, we can efficiently compute directional derivatives, enabling the evaluation of Jacobian-vector products (JVPs) of the form

$$J_g(\mathbf{y})\mathbf{v} = \left. \frac{d}{d\epsilon} g(\mathbf{y} + \epsilon\mathbf{v}) \right|_{\epsilon=0}, \quad (3.5.5)$$

without explicitly forming the Jacobian matrix $J_g(x)$. Furthermore, using the gradient in this framework, we can compute a Hessian-vector product. Both computing the gradient and performing JVP can be done using JAX. Because we can evaluate the Hessian-vector product, we can use power iteration to compute iteratively the leading eigenvalue with its corresponding eigenvector following the algorithm described in Golub and Van Loan (2013). Algorithm 12 depicts the implementation of this method. The methods based on power iteration will be our preferred way of computing the rank-1 approximation; however, as discussed in Golub and Van Loan (2013), power iteration converges as fast as the ratio between the largest and second-largest eigenvalue, which can be very slow if they do not differ sufficiently.

Whether we use SVD or power iteration, it would be difficult to obtain an analytical form of the metric tensor, which is normally required when performing the leapfrog steps. However, with JAX automatic differentiation, we can easily calculate it, making this algorithm usable. JAX will trace all operations in every step of the computation in the algorithm, and it will be able to compute the derivative of each operation and combine them.

When using the power iteration method, we might consider two approaches. On the one hand, we might want to run iterations until convergence, which may be faster for some than others. On the other hand, we might fix the number of iterations, leading to some case where the output is not the best rank-1 matrix we can have. As we will see, once the rank-1 metric is computed, solving the linear systems will become extremely fast, and so the bottleneck will now be computing this matrix. If we were to consider the first approach, we can argue that we will have better samples, as we will get a better approximation of the local geometry. However, if the convergence rate is slow, the algorithm will suffer a significant slowdown. The second approach may then appear more reasonable because it runs the same number of iterations while keeping the computation cost constant for each metric calculation. But while the second method will be faster overall, we would not always get the best rank-1 approximation and make steps that diverge from the target density. Our implementation uses the second method as shown in Algorithm 11 and 12, mainly because the first method is not compatible with JAX just-in-time compilation, and by tuning the number of iterations, we might get a close enough rank-1 approximation at each step.

The real question when using power iteration is which one of those methods will preserve detailed balance. We can argue that as long as the output of the power iteration is deterministic, we will simulate a deterministic Hamiltonian evolution, and detailed balance will be satisfied. In fact, it is no different than choosing a metric other than the Hessian, like the ones discussed in Girolami and Calderhead (2011). If the power iteration is not deterministic, we can argue that the chain will not be reversible because negating the momentum and then simulating the path again from the proposal state does not guarantee that we will return to the starting state. We can enforce this behaviour by fixing the starting point of each power iteration run as done in Algorithm 11 and 12.

Approximating the metric tensor with a low-rank matrix has a major problem because the resultant matrix is singular. To address this issue, we make it positive-definite by adding the low-rank matrix with

a scaled identity matrix and define the rank-1 metric tensor as

$$\hat{\mathbf{G}}(\mathbf{f}) = \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top + \lambda \mathbf{I} \quad (3.5.6)$$

where $\mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top$ is the best rank-1 approximation of the original metric tensor. From this expression we can argue that this is a reasonable choice of metric tensor and that it is an improvement over standard HMC, which uses the identity matrix, by augmenting it with the partial geometric information contained in $\mathbf{u}(\mathbf{f})$ making it a valid halfway point between HMC and RMHMC. In our implementation, we will never have to build this matrix directly. As we will see we can derive expressions which make computation faster throughout the algorithm by taking advantage of the particular form of our metric tensor.

In order to compute the Hamiltonian, we need a way to compute the determinant of the metric tensor. We can compute its determinant using the matrix determinant lemma (see, e.g., [Brookes, 2020](#)),

$$\det(\mathbf{A} + \mathbf{u}\mathbf{v}^\top) = (1 + \mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u}) \det(\mathbf{A}) \quad (3.5.7)$$

which, applied to our metric tensor, gives us,

$$\det \hat{\mathbf{G}}(\mathbf{f}) = \lambda^N \left(1 + \frac{\|\mathbf{u}(\mathbf{f})\|^2}{\lambda}\right) \quad (3.5.8)$$

We can then derive the log-determinant as follows:

$$\log \det \hat{\mathbf{G}}(\mathbf{f}) = N \log \lambda + \log \left(1 + \frac{\|\mathbf{u}(\mathbf{f})\|^2}{\lambda}\right) \quad (3.5.9)$$

This enables a fast log-determinant computation when computing the Hamiltonian in [Algorithm 9](#)

To compute the inverse, we can use the Sherman–Morrison formula (see, e.g., [Golub and Van Loan, 2013](#); [Brookes, 2020](#)),

$$(\mathbf{A} + \mathbf{u}\mathbf{v}^\top)^{-1} = \mathbf{A}^{-1} - \frac{\mathbf{A}^{-1} \mathbf{u} \mathbf{v}^\top \mathbf{A}^{-1}}{1 + \mathbf{v}^\top \mathbf{A}^{-1} \mathbf{u}} \quad (3.5.10)$$

which, applied to our metric tensor, gives us

$$\hat{\mathbf{G}}(\mathbf{f})^{-1} = \frac{1}{\lambda} \mathbf{I} - \frac{1}{\lambda^2} \left(1 + \frac{\|\mathbf{u}(\mathbf{f})\|^2}{\lambda}\right)^{-1} \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top \quad (3.5.11)$$

This is where the advantage of the rank-1 approximation comes in. From the above expression, we can see that solving the linear systems reduces to simply computing the dot product between two vectors and then scaling and shifting the output. In this way computation can be done in $O(N)$. This expression is used throughout [Algorithm 9](#) and [Algorithm 8](#) to enable faster computation compared to RMHMC.

In order to sample, we would want to find $\hat{\mathbf{G}}(\mathbf{f})^{1/2}$ symmetric such that

$$(\hat{\mathbf{G}}(\mathbf{f})^{1/2})(\hat{\mathbf{G}}(\mathbf{f})^{1/2})^\top = (\hat{\mathbf{G}}(\mathbf{f})^{1/2})^2 = \hat{\mathbf{G}}(\mathbf{f}). \quad (3.5.12)$$

Suppose the matrix is of the form $\hat{\mathbf{G}}(\mathbf{f})^{1/2} = c_1 \mathbf{I} + c_2 \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top$. We can write $\hat{\mathbf{G}}(\mathbf{f})$ as follows:

$$\begin{aligned} (\hat{\mathbf{G}}(\mathbf{f})^{1/2})^2 &= (c_1 \mathbf{I} + c_2 \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top)(c_1 \mathbf{I} + c_2 \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top) \\ &= c_1^2 \mathbf{I} + 2c_1 c_2 \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top + c_2^2 \|\mathbf{u}(\mathbf{f})\|^2 \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top \\ (\hat{\mathbf{G}}(\mathbf{f})^{1/2})^2 &= c_1^2 \mathbf{I} + (2c_1 c_2 + c_2^2 \|\mathbf{u}(\mathbf{f})\|^2) \mathbf{u}(\mathbf{f})\mathbf{u}(\mathbf{f})^\top = \hat{\mathbf{G}}(\mathbf{f}) \end{aligned} \quad (3.5.13)$$

Solving for c_1 and c_2 gives us

$$\begin{cases} c_1 &= \sqrt{\lambda} \\ c_2 &= \frac{-\sqrt{\lambda} \pm \sqrt{\lambda + \|\mathbf{u}(\mathbf{f})\|^2}}{\|\mathbf{u}(\mathbf{f})\|^2} \end{cases} \quad (3.5.14)$$

We then have,

$$\hat{\mathbf{G}}(\mathbf{f})^{1/2} = \sqrt{\lambda} \mathbf{I} + \left(\frac{\sqrt{\lambda + \|\mathbf{u}(\mathbf{f})\|^2} - \sqrt{\lambda}}{\|\mathbf{u}(\mathbf{f})\|^2} \right) \mathbf{u}(\mathbf{f}) \mathbf{u}(\mathbf{f})^\top \quad (3.5.15)$$

This expression is used in Algorithm 7 to transform the JAX-generated sample to a momentum variable.

The entire implementation is summarised in Algorithm 7. It follows a similar structure to Algorithm 4 and still shares the accept-reject step (3.3.4) with both RMHMC and HMC.

Algorithm 7 RMHMC with Rank-1 Metric Approximation

Require:

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}_{\text{init}}$: initial position
- 3: t_{max} : number of iterations
- 4: ϵ : leapfrog step size
- 5: τ : number of generalized leapfrog steps
- 6: κ : fixed-point iterations per leapfrog step
- 7: λ : Tikhonov regularisation coefficient

Returns

- 8: $\bar{r}$: rejection rate
- 9: $\{\mathbf{f}^{(t)}\}_{t=1}^{t_{\text{max}}}$: MCMC samples

Initialise

10: $\mathbf{f}^{(1)} \leftarrow \mathbf{f}_{\text{init}}$, rejections $\leftarrow 0$

Begin

```

11: for  $t = 1$  to  $t_{\text{max}}$  do
12:    $\mathbf{u} \leftarrow \text{RankOneMetric}(\log \tilde{p}, \mathbf{f}^{(t)})$  ▷ Rank-1 metric vector (Algorithm 12 or 10)
13:   Sample  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
14:    $\mathbf{p} \leftarrow \sqrt{\lambda} \mathbf{z} + \left( \frac{\sqrt{\lambda + \|\mathbf{u}\|^2} - \sqrt{\lambda}}{\|\mathbf{u}\|^2} \right) (\mathbf{u}^\top \mathbf{z}) \mathbf{u}$  ▷  $\mathbf{p} = \hat{\mathbf{G}}(\mathbf{f}^{(t)})^{1/2} \mathbf{z}$  (3.5.15)
15:    $H \leftarrow \text{HamiltonianRankOne}(\log \tilde{p}, \mathbf{f}^{(t)}, \mathbf{p}, \lambda)$  ▷ Initial Hamiltonian (Algorithm 9)
16:    $(\mathbf{f}', \mathbf{p}') \leftarrow \text{GenLeapfrogRankOne}(\log \tilde{p}, \mathbf{f}^{(t)}, \mathbf{p}, \epsilon, \tau, \kappa, \lambda)$  ▷ New proposal (Algorithm 8)
17:    $H' \leftarrow \text{HamiltonianRankOne}(\log \tilde{p}, \mathbf{f}', \mathbf{p}', \lambda)$  ▷ Proposal Hamiltonian (Algorithm 9)
18:   Sample  $u \sim \text{Uniform}(0, 1)$ 
19:   if  $u < \exp(H - H')$  then ▷ Accept proposal (3.3.4)
20:      $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}'$ 
21:   else ▷ Reject proposal
22:     rejections  $\leftarrow$  rejections + 1
23:      $\mathbf{f}^{(t+1)} \leftarrow \mathbf{f}^{(t)}$ 
24:   end if
25: end for
26:  $\bar{r} \leftarrow \text{rejections} / t_{\text{max}}$ 
27: return  $\bar{r}, \{\mathbf{f}^{(t)}\}$ 

```

End

Algorithm 8 Generalized Leapfrog Integrator (Rank-1 Metric) GenLeapfrogRankOne**Require:**

- 1: $\log \tilde{\mathbf{p}}$: unnormalised log-probability density
- 2: $\mathbf{f}, \mathbf{p}$: current position and momentum
- 3: ϵ : step size; τ : number of steps; κ : fixed-point iterations
- 4: λ : Tikhonov regularisation coefficient

Returns

- 5: $(\mathbf{f}, \mathbf{p}, H)$: updated position and momentum,

Define, $\nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p}) := \frac{\partial}{\partial \mathbf{f}} \text{HamiltonianRankOne}(\log \tilde{\mathbf{p}}, \mathbf{f}, \mathbf{p}, \lambda)$ computed via Autodiff

Begin

```

6: for  $t = 1$  to  $\tau$  do
7:    $\tilde{\mathbf{p}} \leftarrow \mathbf{p}$ 
8:   for  $k = 1$  to  $\kappa$  do                                ▷ Implicit half-step for momentum (3.4.2)
9:      $\tilde{\mathbf{p}} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}, \tilde{\mathbf{p}})$ 
10:  end for
11:   $\mathbf{p} \leftarrow \tilde{\mathbf{p}}$ 
12:   $\mathbf{u} \leftarrow \text{RankOneMetric}(\log \tilde{\mathbf{p}}, \mathbf{f})$                 ▷ Compute the metric (Algorithm 12 or 10)
13:   $\mathbf{w} \leftarrow \frac{1}{\lambda} \mathbf{p} - \frac{1}{\lambda} \left( \frac{\mathbf{u}^\top \mathbf{p}}{\lambda + \|\mathbf{u}\|^2} \right) \mathbf{u}$           ▷  $\mathbf{w} = \hat{\mathbf{G}}(\mathbf{f})^{-1} \mathbf{p}$  (3.5.11)
14:   $\tilde{\mathbf{w}} \leftarrow \mathbf{w}$ 
15:  for  $k = 1$  to  $\kappa$  do                                ▷ Implicit full step for position (3.4.3)
16:     $\tilde{\mathbf{f}} \leftarrow \mathbf{f} + \frac{\epsilon}{2} (\mathbf{w} + \tilde{\mathbf{w}})$ 
17:     $\tilde{\mathbf{u}} \leftarrow \text{RankOneMetric}(\log \tilde{\mathbf{p}}, \tilde{\mathbf{f}})$           ▷ Refresh metric at  $\tilde{\mathbf{f}}$  (Algorithm 12 or 10)
18:     $\tilde{\mathbf{w}} \leftarrow \frac{1}{\lambda} \mathbf{p} - \frac{1}{\lambda} \left( \frac{\tilde{\mathbf{u}}^\top \mathbf{p}}{\lambda + \|\tilde{\mathbf{u}}\|^2} \right) \tilde{\mathbf{u}}$           ▷  $\tilde{\mathbf{w}} = \hat{\mathbf{G}}(\tilde{\mathbf{f}})^{-1} \mathbf{p}$  (3.5.11)
19:  end for
20:   $\mathbf{f} \leftarrow \tilde{\mathbf{f}}$ 
21:   $\mathbf{p} \leftarrow \mathbf{p} - \frac{\epsilon}{2} \nabla_{\mathbf{f}} H(\mathbf{f}, \mathbf{p})$                 ▷ Explicit half-step for momentum (3.4.4)
22: end for
23: return  $(\mathbf{f}, \mathbf{p})$ 

```

End

Algorithm 9 Hamiltonian (Rank-1 Metric) HamiltonianRankOne**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}, \mathbf{p}$: position and momentum
- 3: λ : Tikhonov regularisation coefficient

Returns

- 4: H : Hamiltonian value

$$H(\mathbf{f}, \mathbf{p}) = \frac{1}{2} \mathbf{p}^\top \hat{\mathbf{G}}(\mathbf{f})^{-1} \mathbf{p} + \frac{1}{2} \log \det \hat{\mathbf{G}}(\mathbf{f}) - \log \tilde{p}(\mathbf{f})$$

Begin

- 5: $\mathbf{u} \leftarrow \text{RankOneMetric}(\log \tilde{p}, \mathbf{f})$ ▷ Rank-1 metric vector (Algorithm 12, 11 or 10)
- 6: $\mathbf{w} \leftarrow \frac{1}{\lambda} \mathbf{p} - \frac{1}{\lambda} \left(\frac{\mathbf{u}^\top \mathbf{p}}{\lambda + \|\mathbf{u}\|^2} \right) \mathbf{u}$ ▷ $\mathbf{w} = \hat{\mathbf{G}}(\mathbf{f})^{-1} \mathbf{p}$ (3.5.11)
- 7: $\log \det \hat{\mathbf{G}} \leftarrow N \log \lambda + \log \left(1 + \frac{\|\mathbf{u}\|^2}{\lambda} \right)$ ▷ Rank-1 log-determinant (3.5.9)
- 8: $H \leftarrow \frac{1}{2} \mathbf{p}^\top \mathbf{w} + \frac{1}{2} \log \det \hat{\mathbf{G}} - \log \tilde{p}(\mathbf{f})$
- 9: **return** H

End**Algorithm 10** Rank-1 Metric Approximation (SVD) RankOneMetric**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}$: current position

Returns

- 3: $\mathbf{u} \in \mathbb{R}^N$: rank-1 metric vector such that $\mathbf{G}(\mathbf{f}) \approx \hat{\mathbf{G}}(\mathbf{f}) = \lambda \mathbf{I} + \mathbf{u} \mathbf{u}^\top$

Begin

- 4: $\mathbf{H} \leftarrow \nabla^2(-\log \tilde{p})(\mathbf{f})$ ▷ Full Hessian via Autodiff
- 5: $[\mathbf{V}, \boldsymbol{\sigma}, _] \leftarrow \text{SVD}(\mathbf{H})$ ▷ $\mathbf{H} = \mathbf{V} \text{diag}(\boldsymbol{\sigma}) \mathbf{V}^\top$
- 6: $\sigma_1 \leftarrow \boldsymbol{\sigma}[0]; \mathbf{v}_1 \leftarrow \mathbf{V}[:, 0]$ ▷ Dominant singular value and corresponding eigenvector
- 7: **return** $\mathbf{u} \leftarrow \sqrt{\sigma_1} \mathbf{v}_1$ ▷ $\mathbf{u} \mathbf{u}^\top = \sigma_1 \mathbf{v}_1 \mathbf{v}_1^\top$: rank-1 spectral approximation

End

Algorithm 11 Rank-1 Metric Approximation (Power iteration) RankOneMetricAlt0**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}$: current position
- 3: n_{pow} : number of power iterations

Returns

- 4: $\mathbf{u} \in \mathbb{R}^N$: rank-1 metric vector such that $\mathbf{G}(\mathbf{f}) \approx \hat{\mathbf{G}}(\mathbf{f}) = \lambda \mathbf{I} + \mathbf{u}\mathbf{u}^\top$

Initialise

- 5: $\mathbf{H} \leftarrow \nabla^2(-\log \tilde{p})(\mathbf{f})$ ▷ Full Hessian via Autodiff
- 6: $\mathbf{v} \leftarrow \mathbf{1}_N / \|\mathbf{1}_N\|$ ▷ Normalised all-ones vector

Begin

- 7: **for** $k = 1$ to n_{pow} **do** ▷ *Power iteration* loop
- 8: $\mathbf{v} \leftarrow \mathbf{H}\mathbf{v}$ ▷ Hessian-vector product
- 9: $\mathbf{v} \leftarrow \mathbf{v} / \|\mathbf{v}\|$ ▷ Normalise
- 10: **end for**
- 11: $\sigma_1 \leftarrow \mathbf{v}^\top \mathbf{H} \mathbf{v}$ ▷ Rayleigh quotient: top eigenvalue estimate
- 12: **return** $\mathbf{u} \leftarrow \sqrt{|\sigma_1|} \mathbf{v}$ ▷ $\mathbf{u}\mathbf{u}^\top = |\sigma_1| \mathbf{v}\mathbf{v}^\top$: rank-1 spectral approximation

End**Algorithm 12** Rank-1 Metric Approximation (Power iteration + JVP) RankOneMetricAlt1**Require:**

- 1: $\log \tilde{p}$: unnormalised log-probability density
- 2: $\mathbf{f}$: current position
- 3: n_{pow} : number of power iterations

Returns

- 4: $\mathbf{u} \in \mathbb{R}^N$: rank-1 metric vector such that $\mathbf{G}(\mathbf{f}) \approx \hat{\mathbf{G}}(\mathbf{f}) = \lambda \mathbf{I} + \mathbf{u}\mathbf{u}^\top$

Initialise

- 5: $\mathbf{v} \leftarrow \mathbf{1}_N / \|\mathbf{1}_N\|$ ▷ Normalised all-ones vector

Begin

- 6: **for** $k = 1$ to n_{pow} **do** ▷ *Power iteration* loop
- 7: $\mathbf{v} \leftarrow \text{JVP}(\nabla(-\log \tilde{p}), \mathbf{f}, \mathbf{v})$ ▷ Hessian-vector product via Jacobian-vector product
- 8: $\mathbf{v} \leftarrow \mathbf{v} / \|\mathbf{v}\|$ ▷ Normalise
- 9: **end for**
- 10: $\sigma_1 \leftarrow \mathbf{v}^\top \text{JVP}(\nabla(-\log \tilde{p}), \mathbf{f}, \mathbf{v})$ ▷ Rayleigh quotient: top eigenvalue estimate
- 11: **return** $\mathbf{u} \leftarrow \sqrt{|\sigma_1|} \mathbf{v}$ ▷ $\mathbf{u}\mathbf{u}^\top = |\sigma_1| \mathbf{v}\mathbf{v}^\top$: rank-1 spectral approximation

End

3.6 MCMC diagnostic

The quality of our estimator is heavily dependent on the quality of the samples used to create it. Being able to assess the quality of a set of MCMC samples is as important as the algorithms that produce them. It would be useful to have access to tools that allow us to assess the uncertainty of an estimator constructed from the generated samples. In this section, we will cover the main tool we used to evaluate the output of the MCMC algorithms.

Variance of MCMC Estimators

As stated at the beginning of this chapter, for M independent samples from a distribution, the variance of the Monte Carlo estimator $\bar{\Phi}$ of $\Phi = \mathbb{E}[\phi(\mathbf{f})]$ is $\text{Var}(\phi)/M$. In MCMC, however, successive samples are not independent; they are correlated due to the Markov chain property. To quantify this correlation, we define the lag- k autocorrelation of ϕ along the chain as

$$\rho_k = \frac{\text{Cov}(\phi(\mathbf{f}^{(t)}), \phi(\mathbf{f}^{(t+k)}))}{\text{Var}(\phi)} \quad (3.6.1)$$

For a stationary Markov chain, this quantity does not depend on t , with $\rho_0 = 1$; and as we can see in [Brooks et al. \(2011\)](#), the variance of the estimator for correlated samples can be expressed as

$$\text{Var}(\bar{\Phi}) = \frac{\text{Var}(\phi)}{M} \left(1 + 2 \sum_{k=1}^{\infty} \rho_k \right) \quad (3.6.2)$$

The autocorrelation terms increase the estimator relative to the independent case. This suggests that it can be used to assess the quality of a set of MCMC samples in comparison to independent samples. A high autocorrelation could indicate slow convergence, resulting in a less reliable estimator.

Effective Sample Size

In our work we will evaluate the quality of a set of samples using the effective sample size (ESS), which is defined as (see, e.g., [Robert and Casella, 2004](#))

$$\text{ESS} = \frac{M}{\gamma_{\text{int}}} = \frac{M}{1 + 2 \sum_{k=1}^{\infty} \rho_k} \quad (3.6.3)$$

We can interpret it as the number of independent samples that would yield the same estimator variance as the M correlated MCMC samples.

In practice, the infinite sum defining γ_{int} must be truncated. Our implementation follows the truncation rule proposed in [Geyer \(1992\)](#) where we first group the empirical autocorrelations into consecutive pairs,

$$\Gamma_k = \rho_{2k} + \rho_{2k+1}, \quad k = 0, 1, 2, \dots \quad (3.6.4)$$

As discussed in [Geyer \(1992\)](#), for a reversible Markov chain, the sequence $\{\Gamma_k\}$ is non-negative and non-increasing, and we can rewrite the integrated autocorrelation time as

$$\tau_{\text{int}} = -1 + 2 \sum_{k=0}^{\infty} \Gamma_k \quad (3.6.5)$$

The sum will be truncated at the first index $K + 1$ that violates those properties, ensuring that the sequence $(\hat{\Gamma}_k)_{k \in K}$ is non-negative and non-increasing, where $\hat{\Gamma}_k$ denotes the empirical estimate of Γ_k . This gives

$$\hat{\tau}_{\text{int}} = -1 + 2 \sum_{k=0}^K \hat{\Gamma}_k \quad (3.6.6)$$

Those empirical estimates can be efficiently computed using the fast Fourier transform (FFT) (see, e.g., [Press, 2007](#)).

The above ESS computation is only valid in univariate cases. To handle multivariate samples, we simply compute ESS for each variable and report the minimum. Furthermore, to make a fair comparison between the algorithms, we must consider the amount of time required to run them and incorporate it in our evaluation metric.

4. Experiments

4.1 Preliminaries: Sampling from a 2D Gaussian

To illustrate the difference between the algorithms, we consider the simple case of sampling from a two-dimensional Gaussian distribution, especially when variables are highly correlated with a relatively large difference in variance, which results in a thin-shaped distribution. In this experiment, the initial point, the number of leapfrog steps and the step size are all fixed in order to observe the difference between the dynamics of the algorithms. In particular, the starting point was selected intentionally to demonstrate the main problem of HMC when sampling from a distribution with neither correlation nor scale taken into account. It was those cases that motivated the introduction of RMHMC, which is able to capture that information without having to tune the momentum covariance. Naturally, our Rank-1 RMHMC was also able to capture that information. Our aim here is not to compare the algorithms' performance yet but to show how RMHMC and Rank-1 RMHMC might cope with the case of high correlation and high scale difference, where HMC is known to have trouble. We can see in Figure 4.1a that HMC is very slow at exploring the distribution space compared to RMHMC and Rank-1 RMHMC as shown in Figures 4.1b and 4.1c respectively. In this particular handpicked scenario, the Rank-1 RMHMC proved to be able to improve upon HMC, which further confirms its potential.

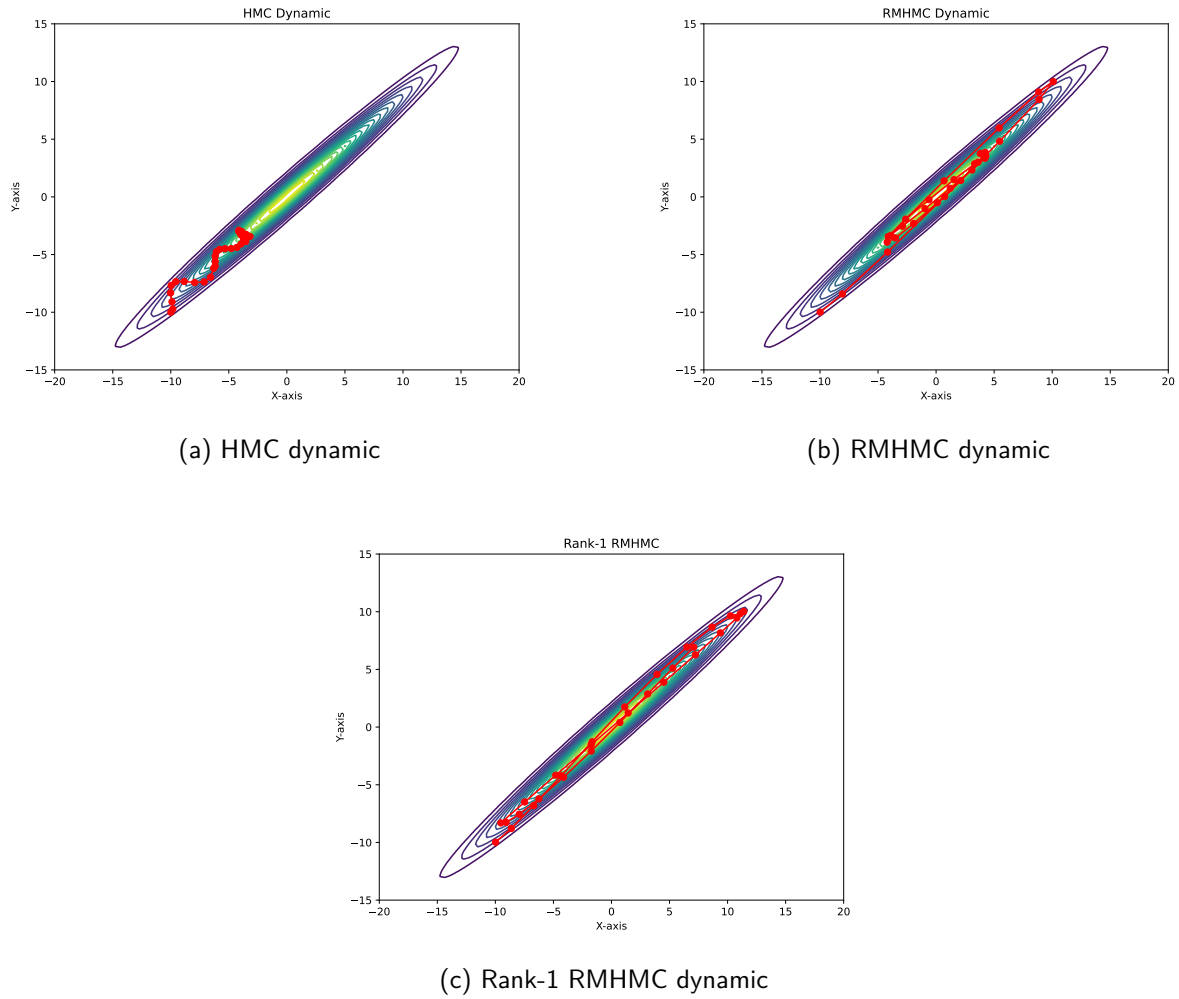


Figure 4.1: MCMC algorithm comparison on a 2D Gaussian

4.2 Experiment Setup

In this chapter, we want to compare RMHMC against Metropolis-Hastings and HMC as well as evaluate Rank-1 RMHMC. As mentioned in previous chapters, we will use the effective sample size (ESS) in order to estimate the quality of the samples. More specifically, we will calculate the univariate effective sample size for each variable and report the minimum. We also consider the algorithm running time in our comparison by computing the ESS per second of each generated chain.

We will run all algorithms on three sets of 30 generated points that will be used to constitute the likelihoods. Each set is generated by taking points from a random sinusoidal time series and then adding Student's t noise with $\mu = 0$, $s = 0$, and $\nu = 2$, which has a heavy tail and will produce outliers in our dataset. The number of points was chosen arbitrarily, but it was deemed sufficiently high-dimensional to observe the difference in running time and sampling capabilities between the three algorithms. We chose to generate the points with sinusoidal functions because the corresponding time series will be stationary, allowing us to ignore the mean term in our prior and only focus on tuning the covariance function.

Our prior will be defined using a Radial Basis Function kernel (2.2.8) as a covariance function. The hyperparameter θ will be tuned for each set using grid search on a set of candidates by evaluating the Monte Carlo estimator in Equation (2.3.9). Our prior distribution will be set to a Student's t -distribution with the same parameter used for data generation. Because both distributions match, we can expect to recover the underlying time series. Information from the prior is incorporated in two ways in the experiment. The first approach is to use the standard posterior distribution obtained directly from the product of the prior and the likelihood. The second relies on reparameterisation via a linear transformation. In this version, we first draw samples from another posterior defined as the product of a standard Gaussian distribution and the likelihood and then transform these samples by multiplying them by the covariance matrix. This procedure still yields the desired samples and is a common strategy to improve the performance of MCMC (see, e.g., Neal, 2012).

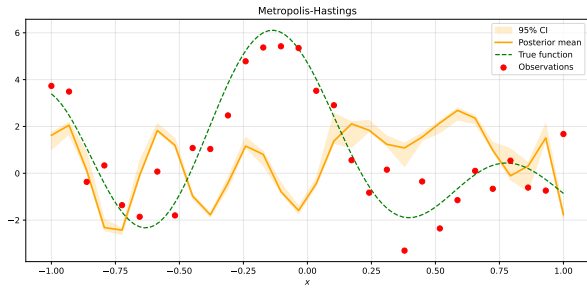
Each algorithm will be run on multiple hyperparameters and generate 5000 samples each. The first half will be discarded (burn-in), and we will only keep every fifth sample, reducing the total size to 500. In the following sections, we will present the results from the best runs, i.e., those with the highest ESS. We first present the results for MH, HMC and RMHMC and then present the result of Rank-1 RMHMC in a separate section. The work is still exploratory, and we do not yet have a guideline as to how to tune the algorithm, and furthermore, although most operation are reduced to $O(N)$, the implementation is still relatively slow due to reasons we will discuss at the end of the chapter.

4.3 MH, HMC, RMHMC Results

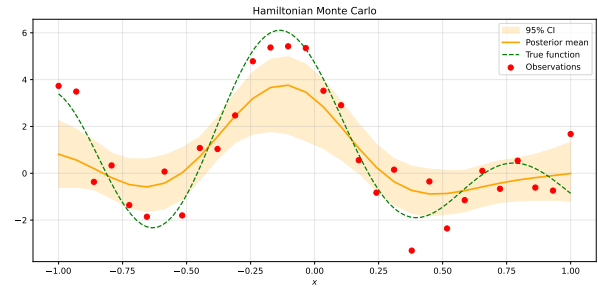
The results of the experiment with the standard posterior sampling are summarised in Table 4.1. We find RMHMC to be much better than MH and HMC in terms of effective sample size in all three datasets. However, when looking at ESS/s, RMHMC is still superior, but its difference with HMC is not that significant anymore. In particular, in the case of MH, we have ESS/s much higher than ESS. This means that the algorithm was running much faster than HMC and RMHMC. However, when we look at the predictive means and confidence intervals in Figures 4.2, 4.3 and 4.4, the predictive mean of MH is very different, while RMHMC and HMC look similar and seem to have partly recovered the true time series. This suggests that unlike RMHMC and HMC, which managed to converge to the stationary distribution, MH, even after 5000 samples and rounds of hyperparameter tuning, has yet to converge, which highlights the ability of both HMC and RMHMC to sample efficiently from a complex distribution.

Table 4.1: MCMC Results

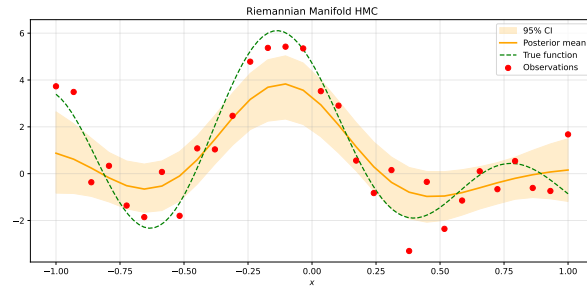
Dataset	Algorithm	ESS	ESS/s
0	MH	5.82	22.15
0	HMC	24.02	4.70
0	RMHMC	500.00	6.15
1	MH	5.82	14.39
1	HMC	13.38	2.70
1	RMHMC	500.00	6.28
2	MH	4.07	17.65
2	HMC	18.66	3.75
2	RMHMC	500.00	6.19



(a) MH - Dataset 0

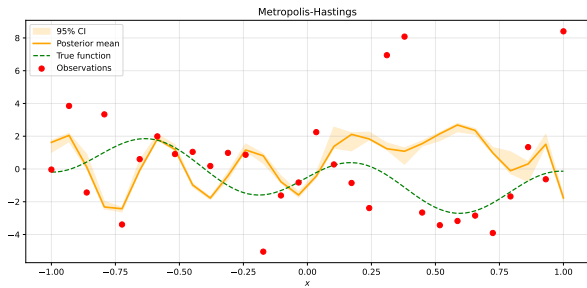


(b) HMC - Dataset 0

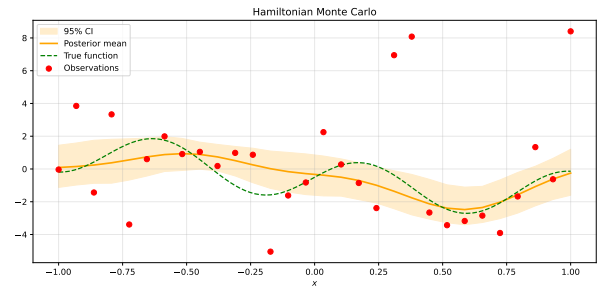


(c) RMHMC - Dataset 0

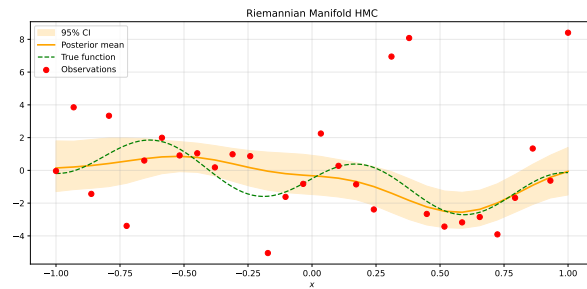
Figure 4.2: MCMC algorithm comparison on Dataset 0.



(a) MH - Dataset 1

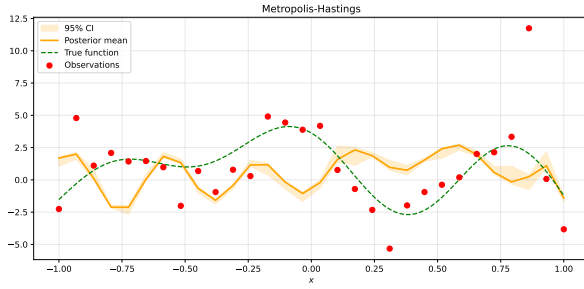


(b) HMC - Dataset 1

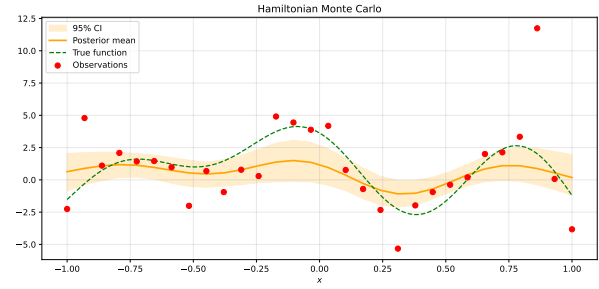


(c) RMHMC - Dataset 1

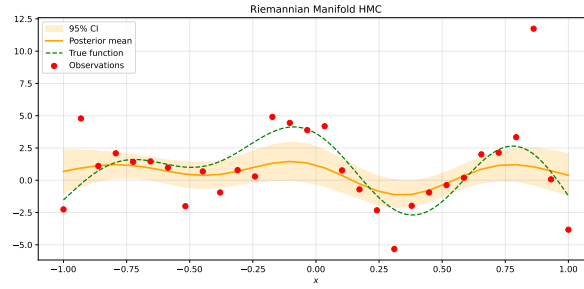
Figure 4.3: MCMC algorithm comparison on Dataset 1.



(a) MH - Dataset 2



(b) HMC - Dataset 2



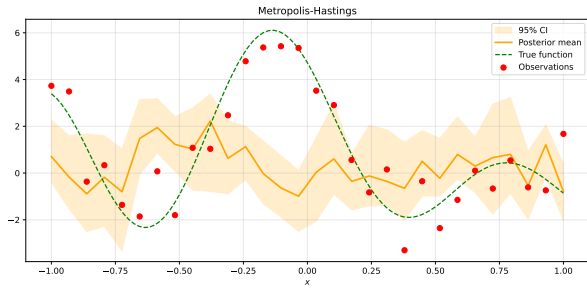
(c) RMHMC - Dataset 2

Figure 4.4: MCMC algorithm comparison on Dataset 2.

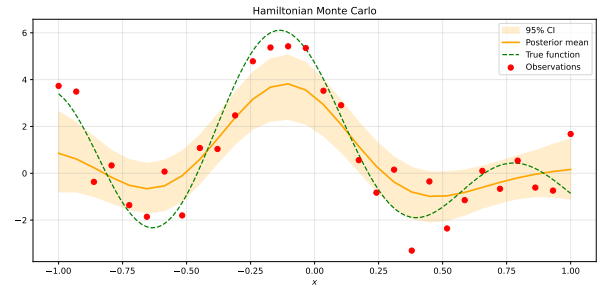
The results for the reparameterised experiment are summarised in Table 4.2. Again, in terms of ESS, RMHMC is much better than MH and HMC. However, HMC has improved a lot in terms of ESS, and when looking at ESS/s, it became far superior to the other algorithms. This suggests that reparameterisation has been more beneficial to HMC than it has been for MH and HMC. Additionally, as shown in Figures 4.5, 4.6 and 4.7, MH still seems to not have converged to the stationary distribution even if it seems to be more able to explore the distribution than in the standard version of the experience.

Table 4.2: Reparameterised MCMC Results

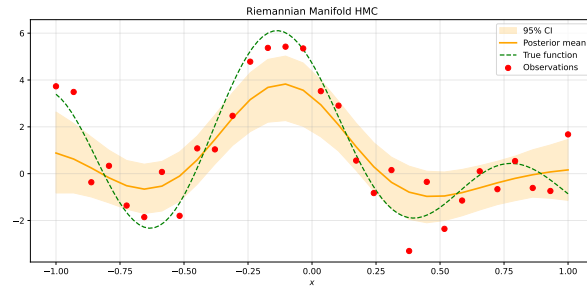
Dataset	Algorithm	ESS	ESS/s
0	MH	4.37	17.69
0	HMC	450.40	205.80
0	RMHMC	500.00	5.22
1	MH	4.60	16.60
1	HMC	441.00	204.00
1	RMHMC	500.00	5.22
2	MH	4.01	12.52
2	HMC	474.20	211.20
2	RMHMC	500.00	5.25



(a) MH - Dataset 0

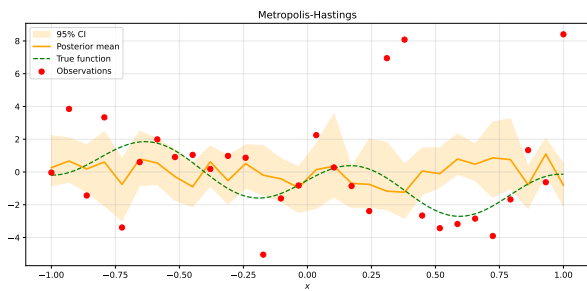


(b) HMC - Dataset 0

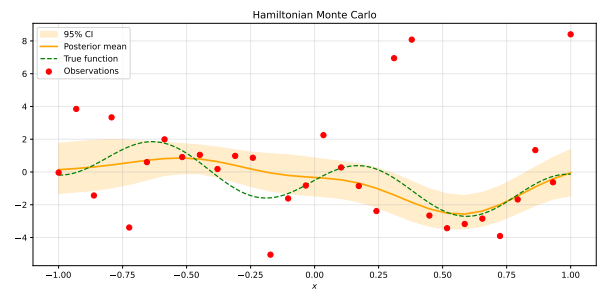


(c) RMHMC - Dataset 0

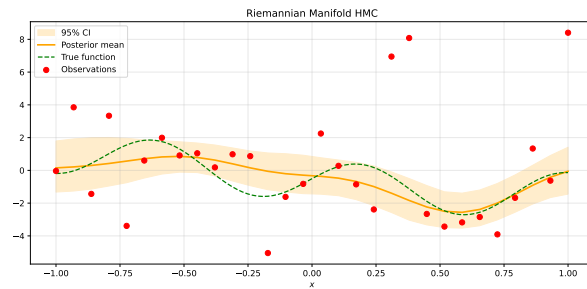
Figure 4.5: Reparameterised MCMC algorithm comparison on Dataset 0.



(a) MH - Dataset 1

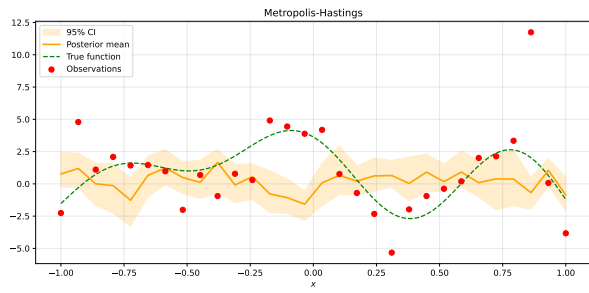


(b) HMC - Dataset 1

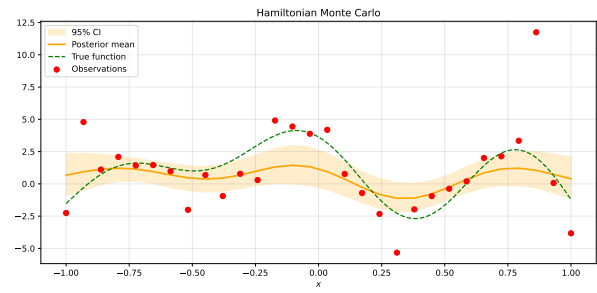


(c) RMHMC - Dataset 1

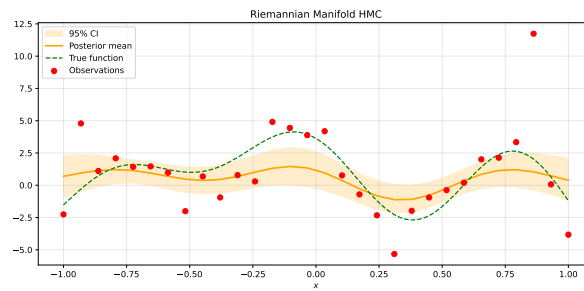
Figure 4.6: Reparameterised MCMC algorithm comparison on Dataset 1.



(a) MH - Dataset 2



(b) HMC - Dataset 2



(c) RMHMC - Dataset 2

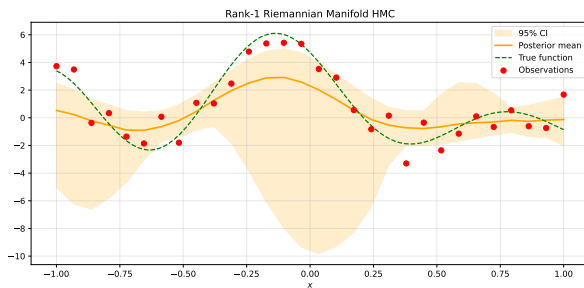
Figure 4.7: Reparameterised MCMC algorithm comparison on Dataset 2.

4.4 Rank-1 RMHMC Results

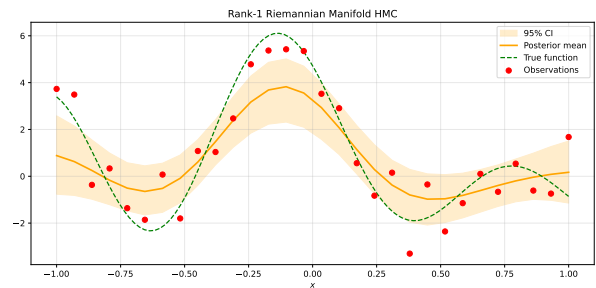
The results of the Rank-1 RMHMC are summarised in Table 4.3. Note that to compute the rank-1 approximation, we used the power iteration approach with 10 iterations. We can see that quantitatively, the algorithm was able to attain reasonable ESS on both algorithms close to HMC. The issue is that we cannot confirm that in the case of the standard experiment, the chains converged, as we can see in Figures 4.8a, 4.9a, 4.10a. More precisely, while the predictive means appear to be close to the true function for each dataset, the issue is that the confidence interval is very large, which might suggest that the algorithm over-explored the support of distribution. When reparameterisation is applied, however, the chain appears to converge as the plots look similar to those of HMC and RMHMC. This can be seen in Figures 4.8b, 4.9b and 4.10b

Table 4.3: Rank-1 RMHMC Results

Dataset	Experiment	ESS
0	Standard	12.54
0	Reparameterised	210.50
1	Standard	13.62
1	Reparameterised	120.50
2	Standard	16.53
2	Reparameterised	312.90



(a) Rank-1 RMHMC - Dataset 0



(b) Reparameterised Rank-1 RMHMC - Dataset 0

Figure 4.8: Rank-1 RMHMC algorithm comparison on Dataset 0.

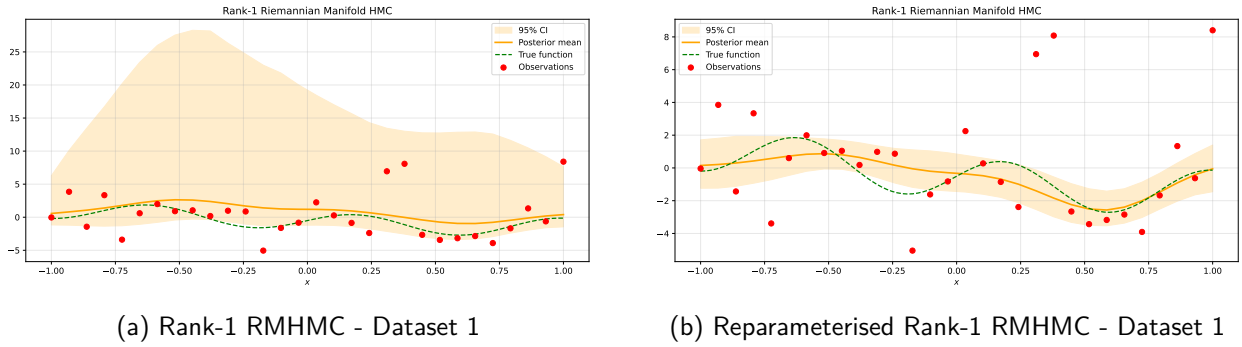


Figure 4.9: MCMC algorithm comparison on Dataset 1.

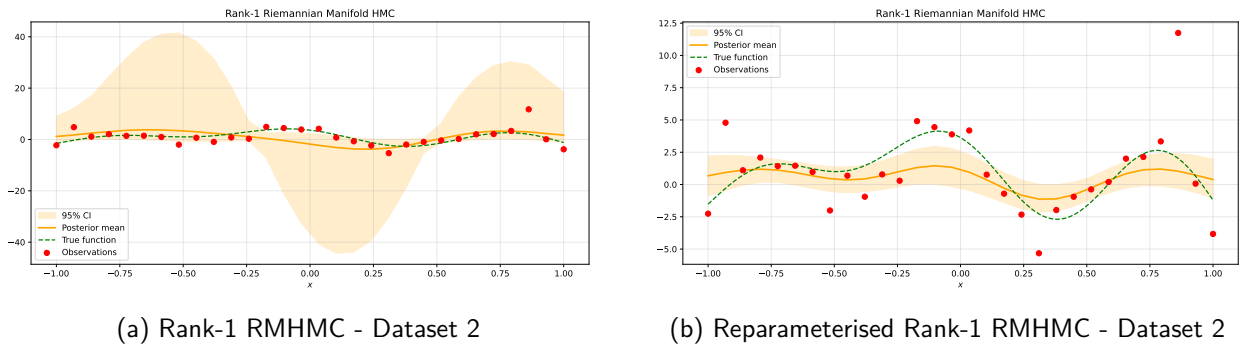


Figure 4.10: Rank-1 RMHMC algorithm comparison on Dataset 2.

4.5 Discussion

In our experiments, we assessed the convergence of the algorithm visually. In this setting, the noise distribution used to generate the data was consistent with the distribution assumed in the model's likelihood. Consequently, when the model is correctly tuned, we expect it to recover the true underlying functions in expectation, and this property enabled us to judge whether the chain had converged. In more complex scenarios, however, convergence could instead be quantified using formal measures.

As shown in our results, RMHMC consistently outperforms its counterparts throughout the entire experiment. The main disadvantage of RMHMC is the computational complexity, which makes each run significantly slower than MH and HMC. However, we see that HMC can largely outperform in ESS/s by reparameterisation, whereas RMHMC seems to benefit little from it.

As discussed in Neal (2012), this reparameterisation through a linear transformation is equivalent to performing HMC with a mass matrix. In a sense, informing the sampler using the prior covariance was enough to incorporate variable scale and correlation information into the sampler. This may explain why RMHMC was consistent across experimental runs since it already includes geometric information in its sampling process and so the reparameterisation did not alter its performance. An interesting question to ask is then whether for other likelihood distributions the prior covariance still contains enough information to improve the sampler.

In the case of Rank-1 RMHMC, we can see that the algorithms still behave poorly. One reason for this could be due to the fact that we only used 10 iterations of power iteration in order to compute the

rank-1 approximation. One could argue that this is the reason why the algorithm seems to over-explore the distribution space, as it may have a poor estimation of the local geometric information at some point and wander off the desired path. The reason we limit ourselves to only 10 iterations, however, is that in order to compute the gradient of the Hamiltonian, automatic differentiation has to unroll all those iterations, and this ends up being computationally more expensive even compared to solving the linear systems in RMHMC. We could also argue that using only rank-1 approximation is the cause of this undesirable behaviour, as it might discard too much geometric information. Using a higher rank could potentially lead to better results. On another note, the regularisation coefficient λ for Equation (3.5.6) might also affect the performance of the algorithm. Throughout the experiments, Rank-1 RMHMC has been tuned only on the number of leapfrog steps and the step size. One may think that tuning this coefficient could improve the behaviour of the algorithm by acting as an approximation to the discarded geometric information. However, for large values of λ , we could suspect that the information brought by the rank-1 metric might become overwritten and the behaviour of the algorithm might go back to being similar to HMC. Even though the sampler performs better and the chain seems to converge when reparameterised, ideally this reparameterisation should reduce the difference in scale and correlation between variables, and therefore, for a reasonable choice of λ we would have a sampler that will be similar to HMC. By introducing the geometric information externally, the metric would not be rich in information anymore and would be negligible compared to $\lambda \mathbf{I}$. However, we can still argue that the fact that this algorithm was better after reparametrisation could confirm that using more information through higher rank can lead to improvement in the results.

5. Conclusion

Throughout this work, we showed that Riemannian manifold Hamiltonian Monte Carlo can efficiently be used to draw quality samples from a complex distribution compared to Metropolis-Hastings and Hamiltonian Monte Carlo. To quantify this, we compared the Effective Sample Size per time of the algorithms while sampling from an intractable posterior distribution defined using a Gaussian process prior and defining the likelihood to be a Student's t-distribution. However, we were able to improve Hamiltonian Monte Carlo using reparameterisation, and it was able to outperform Riemannian manifold Hamiltonian Monte Carlo in our particular case.

Furthermore, we also introduced Rank-1 Riemannian manifold Hamiltonian Monte Carlo, which has the potential of significantly reducing the computational complexity of Riemannian manifold Hamiltonian Monte Carlo. However, it is still underdeveloped and ended up being more computationally expensive than Riemannian manifold Hamiltonian Monte Carlo.

A next step could be to investigate ways to optimise the Rank-1 metric computation so that automatic differentiation does not have to unroll every iteration in order to make gradient computation faster and hence make the algorithm truly usable. We could also investigate ways to automatically tune the regularisation coefficient λ in Equation (3.5.6) to also be a function of $\mathbf{f}$ to better compensate for discarded geometric information. Furthermore, we could expand our experimentation beyond the rank-1 approximation by attempting a higher rank.

Acknowledgements

I would like to thank AIMS for supporting this work. I also extend my gratitude to Google DeepMind for their support through the Google DeepMind Scholarship, and to my supervisor, Prof. Ulrich Paquet, for his invaluable guidance.

References

- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. URL <http://jmlr.org/papers/v18/17-468.html>.
- Box, G. E. P. and Muller, M. E. A note on the generation of random normal deviates. *Annals of Mathematical Statistics*, 29:610–611, 1958.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Katariya, Y., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Brookes, M. The matrix reference manual, 2020. URL <http://www.ee.imperial.ac.uk/hp/staff/dmb/matrix/intro.html>.
- Brooks, S., Gelman, A., Jones, G., and Meng, X.-L. Handbook of markov chain monte carlo. 05 2011. doi: 10.1201/b10905.
- Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., and Riddell, A. Stan: A probabilistic programming language. *Journal of statistical software*, 76(1), 2017.
- Eckart, C. and Young, G. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. doi: 10.1007/BF02288367.
- Geyer, C. J. Practical markov chain monte carlo. *Statistical Science*, 7(4):473–483, 1992. ISSN 08834237. URL <http://www.jstor.org/stable/2246094>.
- Girolami, M. and Calderhead, B. Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 73(2):123–214, 03 2011. ISSN 1369-7412. doi: 10.1111/j.1467-9868.2010.00765.x. URL <https://doi.org/10.1111/j.1467-9868.2010.00765.x>.
- Golub, G. H. and Van Loan, C. F. *Matrix Computations - 4th Edition*. Johns Hopkins University Press, Philadelphia, PA, 2013. doi: 10.1137/1.9781421407944.
- Hastings, W. K. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, Apr 1970. doi: 10.1093/biomet/57.1.97.
- Hayakawa, T. and Asai, S. Fast riemannian-manifold hamiltonian monte carlo for hierarchical gaussian-process models, 2025. URL <https://arxiv.org/abs/2511.06407>.
- Hoffman, M. D. and Gelman, A. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo, 2011. URL <https://arxiv.org/abs/1111.4246>.
- MacKay, D. J. C. *Information theory, inference and learning algorithms*. Cambridge University Press, 2003.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., and Teller, E. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, Jun 1953. doi: 10.1063/1.1699114.

- Neal, R. M. Mcmc using hamiltonian dynamics, 2012. URL <https://arxiv.org/abs/1206.1901>.
- Paquet, U. and Fraccaro, M. An efficient implementation of riemannian manifold hamiltonian monte carlo for gaussian process models, 2018. URL <https://arxiv.org/abs/1810.11893>.
- Pearlmutter, B. A. Fast exact multiplication by the hessian. *Neural Computation*, 6(1):147–160, 1994. doi: 10.1162/neco.1994.6.1.147.
- Phan, D., Pradhan, N., and Jankowiak, M. Composable effects for flexible and accelerated probabilistic programming in numpyro, 2019. URL <https://arxiv.org/abs/1912.11554>.
- Press, W. *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. Numerical Recipes: The Art of Scientific Computing. Cambridge University Press, 2007. ISBN 9780521880688. URL <https://numerical.recipes/book.html>.
- Rasmussen, C. E. and Williams, C. K. I. *Gaussian processes for machine learning*. The MIT Press, 2006.
- Robert, C. P. and Casella, G. Monte carlo statistical methods. In *Springer Texts in Statistics*, 2004.
- Williams, C. and Seeger, M. Using the nystrom method to speed up kernel machines. In Leen, T., Dietterich, T., and Tresp, V., editors, *Advances in Neural Information Processing Systems 13 (NIPS 2000)*, pages 682–688, United States, 2001. MIT Press.